STAFF ENGINEER STUDY GUIDE

Design Patterns

The 23 Gang-of-Four patterns, in plain English — with Java & diagrams

Distilled from *"Design Patterns: Elements of Reusable Object-Oriented Software"* by Gamma, Helm, Johnson & Vlissides (Addison-Wesley, 1994), and enriched with modern research on how these patterns actually appear in the JDK, Spring, and real Java codebases.

5 Creational | 7 Structural | 11 Behavioral | 2 core OO principles

Each pattern: plain-English intent · analogy · problem · Java example · real-world usage · pitfalls · staff-eng lens

Contents

How to read this guide. Start with the two foundational principles — nearly every pattern is an application of one or both. Then work through the three families. Creational patterns are about *making* objects, structural patterns about *composing* them, and behavioral patterns about how they *talk and share responsibility*.

Observer — notify dependents when state changes

State — change behavior when internal state changes

Strategy — swap interchangeable algorithms at runtime

Template Method — fix an algorithm's skeleton, defer steps

Visitor — add operations to a structure without changing it

Reference

Cheat sheet — all 23 patterns, intent & when to use, on one spread

◆ Foundations

Before you memorize 23 pattern names, learn the two ideas the whole book is built on. Almost every pattern in the Gang of Four (GoF) catalog is just one of these two principles applied to a specific problem. If you understand these deeply, you can derive most patterns yourself instead of pattern-matching from memory — which is exactly what a staff engineer is expected to do in an interview.

What is a "design pattern," really?

A **design pattern** is a named, reusable solution to a design problem that keeps showing up in object-oriented software. It is not code you copy-paste — it's a *shape* of a solution, described so precisely that two engineers who've never met can say "let's use an Observer here" and both immediately picture the same structure. That's the real value: patterns are a shared vocabulary that lets teams talk about design at a higher level than individual classes and methods.

KEY IDEA

Every pattern in the book is documented with the same template, and it's worth internalizing this template because it's how you should structure a pattern discussion in an interview:

- **Name** — the vocabulary word (e.g. "Decorator").
- **Intent** — the one-sentence problem it solves.
- **Participants** — the roles/classes involved and how they relate.
- **Consequences** — the tradeoffs: what you gain, what you give up.

Staff-eng lens: interviewers rarely want you to just name a pattern — they want to see you reach the pattern by reasoning from a force ("this creation logic is exploding into if/else" or "this class has five unrelated reasons to change"). Naming the pattern is the destination; showing the reasoning is what's actually being graded.

Principle 1 — Program to an interface, not an implementation

Idea in one sentence: code that depends on a concrete class only knows how to talk to that one class; code that depends on an interface can talk to anything that implements it.

When you write a variable's type as the concrete class, you've welded your code to that one implementation. When you write it as the interface, you've left a socket that any compatible implementation can plug into — including a fake one in a test.

```
// Welded to one implementation — hard to change, hard to test
ArrayList<String> names = new ArrayList<>();
void printAll(ArrayList<String> list) { ... }

// Programmed to an interface — flexible, testable
List<String> names = new ArrayList<>();
void printAll(List<String> list) { ... }

// Now the caller can pass ANY List: ArrayList, LinkedList,
// an immutable List.of(...), or a test double — printAll never changes.
```

Nothing about `printAll`'s logic needed `ArrayList` specifically — it only needed the ability to iterate and get a size. By declaring the parameter as **List**, you've said "I depend on the *contract*, not the *class*." That single change is what makes mocking possible in unit tests: a test can hand `printAll` a hand-rolled fake list, and production code can later swap `ArrayList` for a `CopyOnWriteArrayList` under load, without touching `printAll` at all.

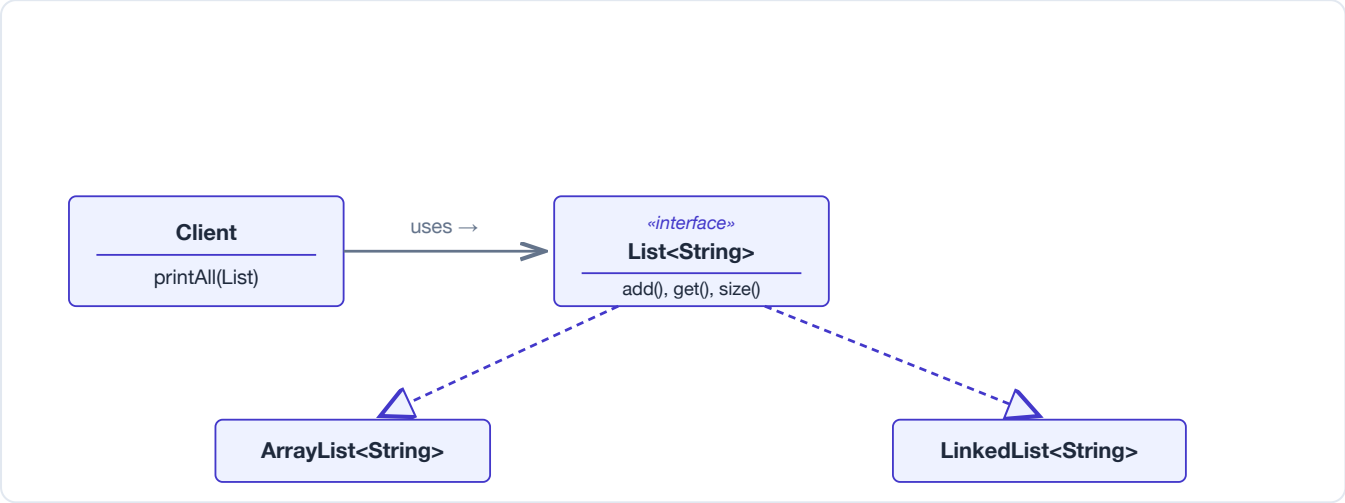

*Figure — The client depends on the **List** interface, not on **ArrayList**. Either concrete implementation (or a test mock) can be substituted without changing the client.*

BENEFITS

Substitutability (swap implementations for performance or behavior), testability (inject a mock/fake), and looser coupling between modules — the client only needs to know the *shape* of the dependency, not its internals.

Principle 2 — Favor object composition over class inheritance

Idea in one sentence: instead of a subclass inheriting a superclass's guts at compile time, give an object a reference to another object and delegate to it — swappable at runtime.

Inheritance is the first tool most engineers reach for when they want to reuse behavior — "I need a Car that can also drive itself, so I'll make a `SelfDrivingCar extends Car`." The GoF book's core warning is that inheritance is a much heavier commitment than it looks:

- **White-box reuse** — the subclass sees and depends on the superclass's internal implementation, not just its public contract.
- **Decided at compile time** — you cannot change what a subclass inherits from once compiled and running.
- **Breaks encapsulation** — a change to the superclass can silently break subclasses ("fragile base class" problem), because the coupling runs deeper than the public API.
- **One dimension only** — single inheritance in Java means one axis of "is-a" per class; combining independent behaviors quickly forces awkward class explosions.

Composition fixes this by making the relationship "has-a" instead of "is-a": one object holds a reference to another (typically through an interface — see Principle 1) and forwards calls to it.

```
// Rigid: a TaxiDriver IS-A Car? That's already a weird sentence.
class TaxiDriver extends Car { ... }

// Flexible: a Driver HAS-A Car, and can swap it at runtime.
interface Car { void drive(); }

class Driver {
    private Car car;                    // composition: a reference, not a base class
    Driver(Car car) { this.car = car; }
    void driveToAirport() { car.drive(); }
    void swapCar(Car newCar) { this.car = newCar; }   // impossible with inheritance
}
```

`Driver` doesn't need to know how `Car` drives internally — it just calls the public `drive()` method. And because `car` is just a field, you can hand the `Driver` a different `Car` implementation (electric, rental, a test stub) at runtime, something a compile-time subclass relationship can never do.

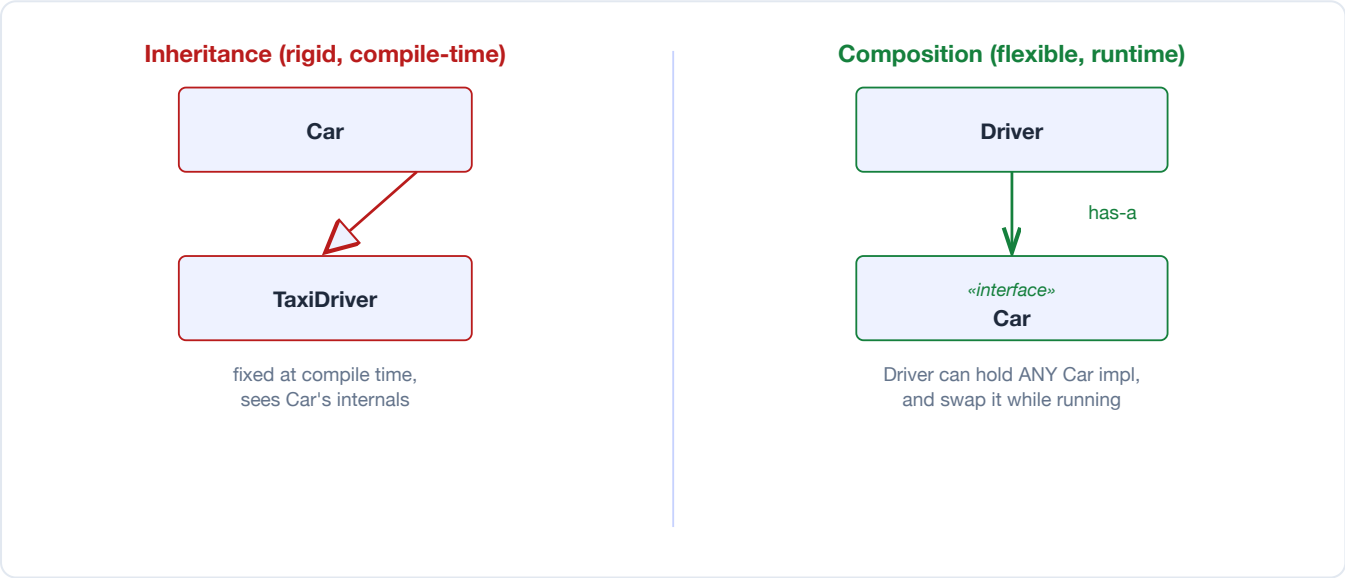

*Figure — Left: **TaxiDriver** is welded to **Car** at compile time. Right: **Driver** holds a reference to the **Car** interface and can swap the concrete car object at runtime.*

Aspect	Inheritance (is-a)	Composition (has-a)
Coupling	Tight — subclass depends on superclass internals	Loose — depends only on the held object's interface
When decided	Compile time (fixed once written)	Runtime (can be reassigned/injected)
Encapsulation	Often broken ("fragile base class")	Preserved — internals of the held object stay hidden
Flexibility	Low — one fixed parent per class	High — swap, stack, or combine multiple collaborators

PITFALLS

This principle does *not* say "never use inheritance." Inheritance is still the right tool when there's a genuine, stable is-a relationship and you want to reuse a contract (e.g. `abstract class` defining a template method). The warning is against using inheritance purely as a shortcut for code reuse when the relationship isn't truly "is-a," or when you need to change behavior at runtime.

Real-world analogy: think of inheritance like being born into a family business — you inherit the whole operation, warts and all, and you can't easily un-inherit it. Composition is like hiring a contractor: you give them a job through a contract (interface), you can fire and rehire different contractors any time, and you never need to know how they do the work internally.

Staff-eng lens: over-using inheritance is the single most common mistake among junior-to-mid engineers — deep class hierarchies that seemed elegant on day one become nearly impossible to change by year two. If you can articulate in an interview *why* you'd choose composition ("I don't want this behavior locked in at compile time, and I don't want subclasses reaching into my internals"), that signals more design maturity than reciting pattern names. Also note: this principle is the DNA of **Strategy**, **Decorator**, **Bridge**, and **State** — all four are really just "favor composition" applied to a specific shape of problem.

MODERN JAVA CHANGES THE MECHANICS, NOT THE PRINCIPLE

Several GoF patterns were designed against 1994-era Java/C++/Smalltalk. Modern Java gives you lighter-weight tools for the same ideas:

- **Interfaces with default methods** let an interface ship partial implementation, reducing the need for abstract base classes.
- **Lambdas / method references** often replace a whole Strategy or Command class hierarchy with a single functional interface (e.g. `Comparator`, `Runnable`).
- **Records** give you a terse, immutable value type — handy for Value Objects, Flyweight keys, and immutable Command payloads.
- **Sealed interfaces** let you close a type hierarchy and pattern-match exhaustively, which changes how you implement Visitor-like double dispatch.

You'll see these called out as "modern Java note" alongside the classic implementation in later chapters.

Patterns at a glance — the three families

Creational patterns are about making object *creation* flexible instead of hard-coded. Plain `new SomeClass()` calls hard-wire your code to one concrete class at the exact point of construction — which fights Principle 1 directly. Creational patterns (Factory Method, Abstract Factory, Builder, Prototype, Singleton) all move the decision of "which concrete class to instantiate, and how" out of the calling code and into one focused place, so the rest of the system only ever depends on interfaces.

Structural patterns are about composing classes and objects into larger structures while keeping them loosely coupled. These patterns (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) answer "how do I wire these objects together — make incompatible interfaces work together, add behavior without subclassing, treat a tree of objects uniformly, hide a messy subsystem behind a simple face" — all without the pieces needing to know much about each other's internals.

Behavioral patterns are about assigning responsibilities between objects and defining how they communicate. Rather than one god-object owning all the logic, these patterns (Strategy, Observer, Command, State, Template Method, Chain of Responsibility, Iterator, Mediator, Memento, Visitor, Interpreter) distribute behavior across collaborating objects and standardize how those objects talk to each other — who decides, who reacts, who gets notified, and in what order.

HOW TO CHOOSE A PATTERN — START FROM THE FORCE, NOT THE NAME

In an interview, don't start by trying to recall a pattern name. Start from the symptom in front of you:

- "*Creating this object is getting complicated*" (many constructor params, need a different subclass per context, need to control how many instances exist) → look at **Creational**.
- "*These two things don't fit together, or this structure needs to be composed/wrapped/simplified*" (incompatible interfaces, want to add behavior without an explosion of subclasses, want to treat a tree uniformly) → look at **Structural**.
- "*This class is doing too much, or objects are too tightly coupled by how they call each other*" (giant if/else on type or state, one object needs to react to another's changes, an algorithm needs to vary) → look at **Behavioral**.

Staff-eng lens: the biggest interview red flag is reaching for a pattern when a plain object, a lambda, or a simple `if` would do — that's the "when all you have is a hammer" failure mode, and senior interviewers watch for it. The strongest answer is often "here's the simplest thing that works, and here's the specific force that would make me introduce pattern X later if it shows up" — showing you know both the pattern *and* its cost.

Creational Patterns

Creational patterns are all about one question: *who decides which concrete class gets instantiated, and how?* A bare `new SomeClass()` hard-wires that decision into whatever code happens to call it, which fights the "program to an interface" principle head-on. The five patterns in this family — Factory Method, Abstract Factory, Builder, Prototype, and Singleton — each move that decision out of the calling code and into one focused place, so the rest of your system only ever depends on interfaces and never has to know (or care) which concrete class showed up.

C Factory Method

Intent (plain English): Define an interface for creating an object, but let subclasses decide which class to instantiate.

Real-world analogy: think of a logistics company's head office. The head office defines "we deliver packages" as a process, but it never says whether that's a truck or a ship — the regional branch (road division vs. sea division) decides which vehicle to use when a delivery request comes in.

PROBLEM IT SOLVES

You have a class whose algorithm is fixed, but one step of that algorithm — "create the object I need to work with" — has to vary depending on context. If you hard-code `new Truck()` inside the shared class, you can never support sea delivery without editing that class. Factory Method pulls the "which class to create" decision into an overridable method, so new product types can be added by writing a new subclass instead of editing existing code.

HOW IT WORKS

An abstract (or interface) "creator" class declares a factory method, e.g. `createTransport()`, and uses only that method's return type — never a concrete class — in its own logic. Concrete subclasses override the factory method to return a specific product. The creator's other methods stay completely unaware of which concrete product they're working with; they just call the interface methods on whatever the factory method handed back.

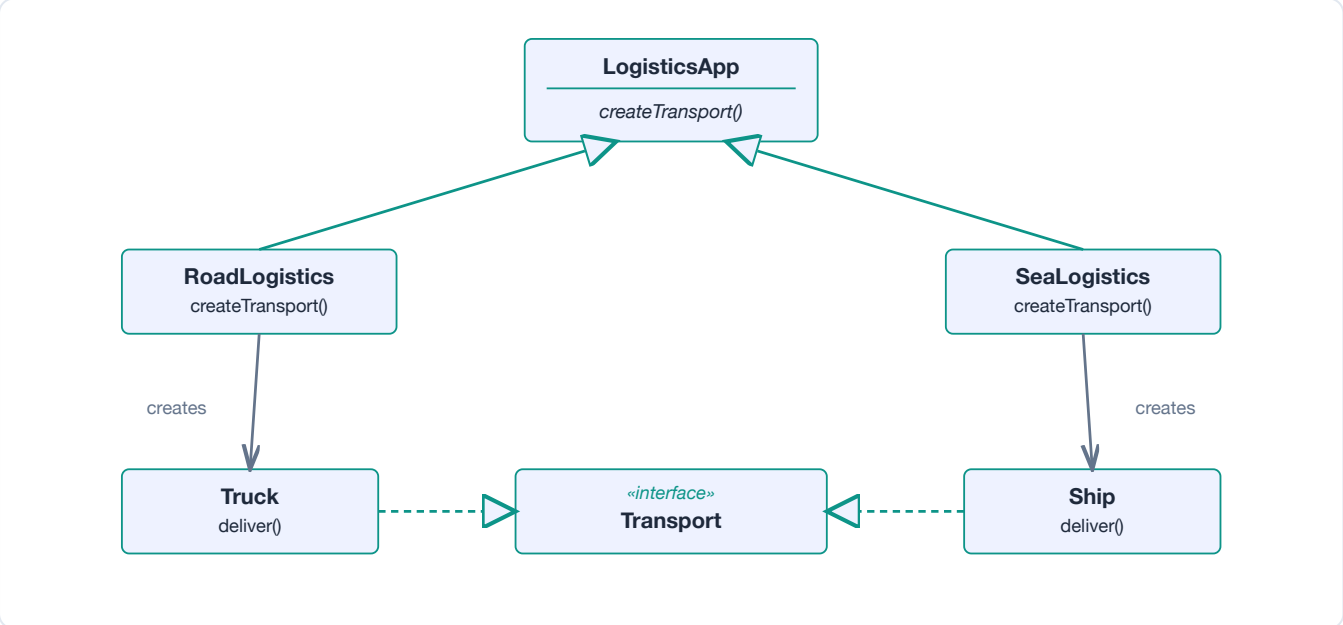

*Figure — Each subclass of **LogisticsApp** overrides **createTransport()** to return its own **Transport** implementation. The rest of **LogisticsApp** never mentions **Truck** or **Ship** by name.*

JAVA EXAMPLE

```
abstract class LogisticsApp {
    abstract Transport createTransport();      // the factory method
    void planDelivery() {
        Transport t = createTransport();        // never names a concrete class
        t.deliver();
    }
}

interface Transport { void deliver(); }
class Truck implements Transport {
    public void deliver() { System.out.println("Deliver by road"); }
}
class Ship implements Transport {
    public void deliver() { System.out.println("Deliver by sea"); }
}

class RoadLogistics extends LogisticsApp {
    Transport createTransport() { return new Truck(); }
}
class SeaLogistics extends LogisticsApp {
    Transport createTransport() { return new Ship(); }
}
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- Spring's `BeanFactory.getBean(...)` — the container decides which concrete implementation to hand you based on configuration.
- `Calendar.getInstance()` — returns a `GregorianCalendar` (or a locale-specific subclass) without you ever naming it.
- `NumberFormat.getInstance()` — returns a locale-appropriate concrete formatter.
- JDBC's `Connection.createStatement()` — returns a driver-specific `Statement` implementation.

PITFALLS & CRITICISMS

It's easy to reach for this before you actually need it. If you only ever expect one concrete product, you've built a class hierarchy — and the ceremony of subclassing just to instantiate one thing — for a variation that may never arrive. That's speculative generality, and it makes the code harder to read for no real benefit.

Staff-eng lens: reach for Factory Method when you can point to at least two concrete products today (or a near-certain second one on the roadmap) and the creating class's other logic genuinely doesn't care which one it got. If there's only one implementation and no credible plan for a second, a plain constructor or a static factory method (`Truck.create()`) is simpler and just as testable — don't build the hierarchy speculatively.

C Abstract Factory

Intent (plain English): Provide an interface for creating families of related or dependent objects without specifying their concrete classes.

Real-world analogy: think of furniture shopping by style, not by piece. If you pick "Scandinavian," every item you order — chair, table, sofa — comes in that one consistent style. You never accidentally end up with a Scandinavian chair next to a Baroque table, because the whole family is produced by the same "Scandinavian factory."

PROBLEM IT SOLVES

Sometimes you don't just need to vary one product (that's Factory Method) — you need to vary a whole *set* of related products together, and guarantee they stay consistent with each other. If a UI toolkit lets you mix a Windows-style button with a Mac-style checkbox, the UI looks broken. Abstract Factory gives you one factory object per "family," so picking the family once guarantees every product it hands out matches.

HOW IT WORKS

You define an abstract factory interface with one creation method per product type in the family (`createButton()`, `createCheckbox()`). Each concrete factory (`WinFactory`, `MacFactory`) implements all of those methods using its own consistent set of concrete product classes. Client code is handed one factory instance and calls its methods — it never touches `new WinButton()` directly, so it can't accidentally mix families.

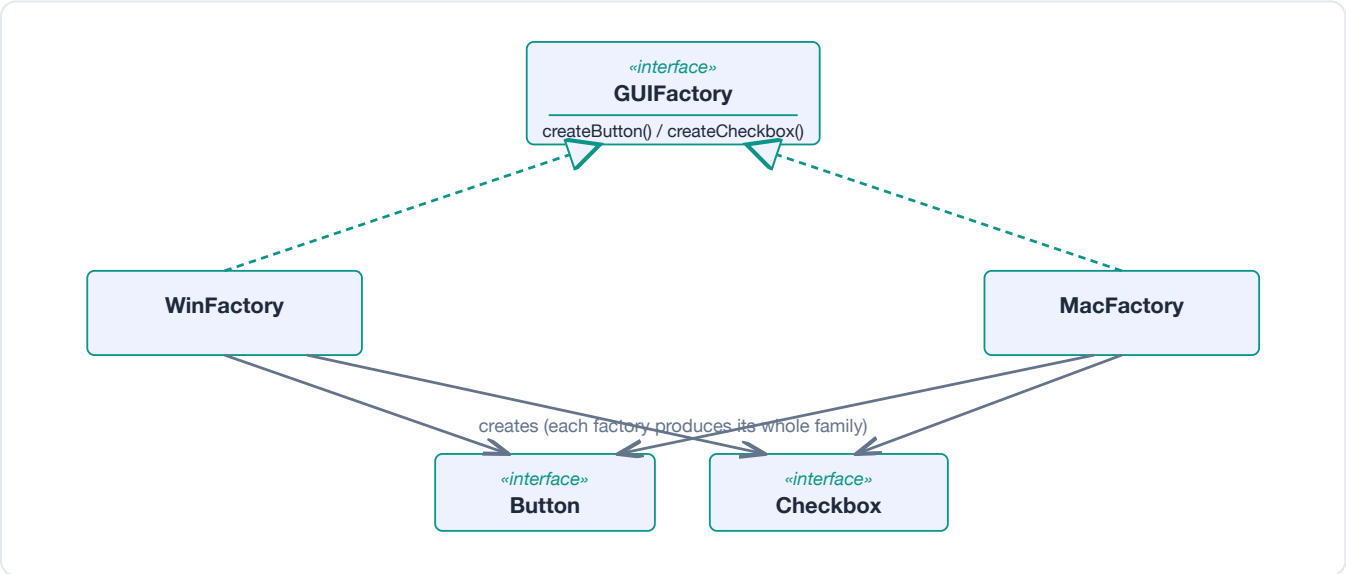

*Figure — **WinFactory** and **MacFactory** each implement **GUIFactory** and produce a matching, internally-consistent **Button** + **Checkbox** pair.*

JAVA EXAMPLE

```
interface Button { void render(); }
interface Checkbox { void render(); }

interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

class WinButton implements Button { public void render() { System.out.println("Win button");
} }
class WinCheckbox implements Checkbox { public void render() { System.out.println("Win
checkbox"); } }

class WinFactory implements GUIFactory {
    public Button createButton() { return new WinButton(); }
    public Checkbox createCheckbox() { return new WinCheckbox(); }
}
// MacFactory mirrors WinFactory, returning MacButton / MacCheckbox instead.

class Application {
    Application(GUIFactory factory) {
        Button button = factory.createButton();     // caller never says "new WinButton()"
        Checkbox checkbox = factory.createCheckbox();
    }
}
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `DocumentBuilderFactory` / `SAXParserFactory` / `TransformerFactory` in `javax.xml.*` — each returns a consistent family of XML-processing objects for the configured provider.
- JDBC drivers — `DriverManager` hands you a vendor-specific `Connection`, and everything it creates (`Statement`, `ResultSet`) is from that same vendor's consistent family.

PITFALLS & CRITICISMS

Adding a brand-new product to the family (say, a `Slider`) means touching the abstract factory interface *and every single concrete factory* — that's a ripple effect Factory Method doesn't have. It's also simply more moving parts than most problems need: don't introduce a whole family abstraction to swap one thing.

Staff-eng lens: the tell that distinguishes this from Factory Method is "must multiple products stay consistent with each other." If mixing-and-matching products would actually break something (a Win button next to a Mac checkbox looks wrong), Abstract Factory earns its complexity. If you're only ever varying one product independently, you don't need the family concept — plain Factory Method (or even a single injected dependency) is enough.

C Builder

Intent (plain English): Separate the construction of a complex object from its representation, so the same construction process can create different representations.

Real-world analogy: ordering a custom pizza. You don't hand the cashier one giant constructor call with fifteen positional arguments ("large, thin crust, no sauce, extra cheese, olives, null, null, mushrooms..."). You say what you want one step at a time — size, then crust, then toppings — and only at the end does the kitchen actually assemble the finished pizza.

PROBLEM IT SOLVES

A class with many optional fields tempts you into a **telescoping constructor** — `new Pizza(size, crust, cheese, olives, mushrooms, ...)` — or a pile of overloaded constructors for every combination. Both are error-prone (easy to swap two same-typed positional args) and don't scale as fields grow. Builder replaces that with readable, step-by-step construction and lets the final object be built immutable.

HOW IT WORKS

A separate `Builder` object exposes one fluent method per optional attribute, each returning `this` so calls can be chained. Once every attribute is set, a single `build()` call assembles the actual (often immutable) target object in one shot, validating everything together instead of leaving it in a half-constructed state along the way.

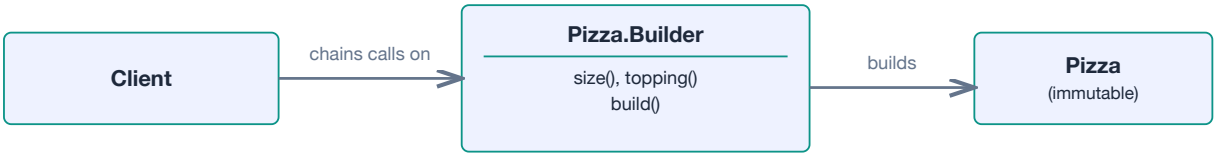

*Figure — the **Client** chains fluent setter calls on **Pizza.Builder**; a single **build()** call produces the final immutable **Pizza**.*

JAVA EXAMPLE

```
public final class Pizza {
    private final String size;
    private final List<String> toppings;

    private Pizza(Builder b) {
        this.size = b.size;
        this.toppings = List.copyOf(b.toppings);   // defensively immutable
    }

    public static class Builder {
        private String size = "medium";
        private final List<String> toppings = new ArrayList<>();

        public Builder size(String size) { this.size = size; return this; }
        public Builder topping(String t) { toppings.add(t); return this; }
        public Pizza build() { return new Pizza(this); }
    }
}

Pizza order = new Pizza.Builder()
    .size("large")
    .topping("olives")
    .topping("feta")
    .build();
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `StringBuilder` — the canonical JDK builder, chaining `append()` calls before a final `toString()`.
- `java.time.*` builders, e.g. `Locale.Builder`.
- Lombok's `@Builder` annotation — generates exactly this pattern for you.
- `Stream.Builder` and `HttpRequest.newBuilder()` (Java's HTTP client) — both fluent, immutable-result builders.

PITFALLS & CRITICISMS

For a class with two or three fields, a Builder is pure ceremony — a constructor or a static factory method (`Pizza.of(size, toppings)`) reads just as clearly with a fraction of the boilerplate. Don't add a Builder "because it's best practice"; add it when the constructor argument list is genuinely getting unwieldy.

Staff-eng lens: the deciding signal is optional-parameter explosion or a need to validate the whole object atomically before it exists (no partially-constructed instances visible to other threads). If Lombok or a records-based static factory already covers your case in one line, don't hand-roll a Builder just for the name recognition in an interview — showing you know *when not to* use it is the stronger signal.

C Prototype

Intent (plain English): Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.

Real-world analogy: a photocopier. Instead of typing out a new document from scratch every time (calling a constructor and re-setting every field), you put an existing, already-filled-out document on the glass and press copy. The copy starts as an exact duplicate you can then tweak.

PROBLEM IT SOLVES

Sometimes building an object from scratch is expensive (it was loaded from a database, computed from a slow algorithm, or configured through a long chain of setup calls) or the exact class to instantiate is only known at runtime as an existing object, not as a compile-time type name. Prototype lets you get a new, independent object by copying a template instance instead of re-running that expensive or type-unknown construction.

HOW IT WORKS

Every participating class implements a `clone()` method that knows how to duplicate itself. A client that holds a reference to some prototype object — often through a common interface, without knowing its concrete class — just calls `clone()` to get a new, independent instance instead of calling `new` and re-populating every field by hand.

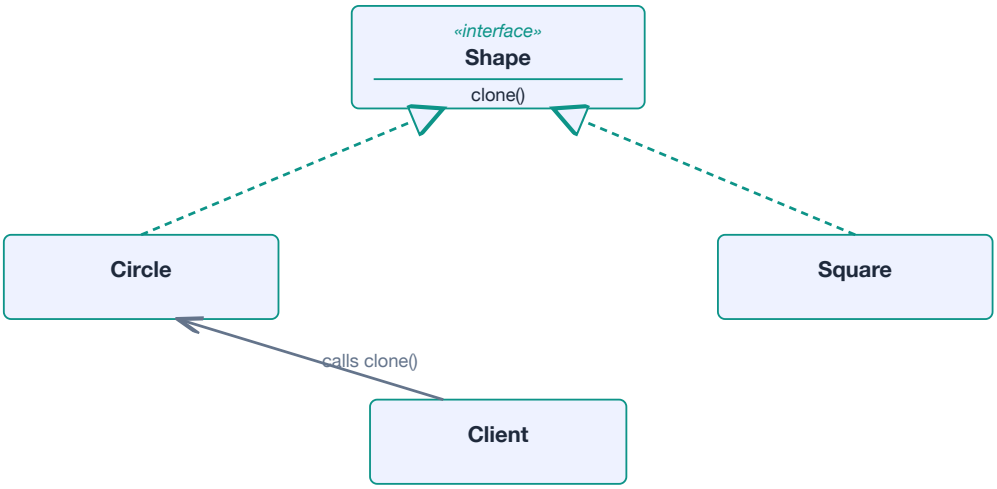

*Figure — the **Client** asks an existing **Circle** instance to **clone()** itself, getting a new independent object without knowing its concrete constructor.*

JAVA EXAMPLE

```
abstract class Shape implements Cloneable {
    int x, y;
    abstract void draw();

    @Override
    public Shape clone() {
        try {
            return (Shape) super.clone();     // shallow copy of all fields
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);       // can't happen: we implement Cloneable
        }
    }
}

class Circle extends Shape {
    int radius;
    void draw() { System.out.println("circle r=" + radius); }
}

Circle original = new Circle();
original.radius = 10;
Circle copy = (Circle) original.clone();   // new object, same field values
copy.radius = 20;                          // original is untouched
```

SHALLOW VS. DEEP COPY — THE CAVEAT THAT ALWAYS COMES UP IN INTERVIEWS

`super.clone()` only does a **shallow copy**: primitive fields are duplicated, but reference fields (like a `List` or another object) still point at the *same* underlying object as the original. If `Shape` held a mutable `List<Point>`, mutating that list through the copy would silently mutate the original too. A correct deep-copying `clone()` must explicitly clone those mutable fields as well.

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `Object.clone()` / the `Cloneable` marker interface — Java's built-in (if famously clunky) version of this pattern.
- Spring's `prototype`-scoped beans — the container keeps a template bean definition and produces a fresh instance on every request, conceptually the same idea at a framework level.

PITFALLS & CRITICISMS

Joshua Bloch's *Effective Java* flags `Cloneable` itself as broken: it has no `clone()` method of its own, relies on a checked exception that shouldn't be checked, and defaults to shallow copying that silently causes bugs when fields are mutable. Most modern Java codebases prefer an explicit **copy constructor** or a static `copyOf(...)` factory method over implementing `Cloneable`.

Staff-eng lens: use Prototype when construction is genuinely expensive or the concrete type is only known via an existing runtime instance. If you're just copying a simple data object, skip `Cloneable` entirely and write a copy constructor — it's clearer, avoids the checked-exception dance, and makes deep vs. shallow behavior explicit at the call site instead of hidden behind an override.

C Singleton

Intent (plain English): Ensure a class only has one instance, and provide a global point of access to it.

Real-world analogy: a country's central bank. There's deliberately only one, every part of the economy refers to the same one, and you can't just spin up a second one whenever you want a different interest rate.

PROBLEM IT SOLVES

Some resources genuinely only make sense as one instance per process — a single configuration object, a single connection pool, a single logging registry. Singleton guarantees exactly one instance exists and gives every part of the program a way to reach it, instead of trusting every caller to remember "don't construct a second one."

HOW IT WORKS

The class hides its constructor (private) and exposes the single instance through a static accessor. In Java, the cleanest and safest way to do this is an `enum` with one value — the language itself guarantees the instance is created exactly once, is thread-safe, and even survives serialization correctly, none of which classic hand-written singletons get for free.

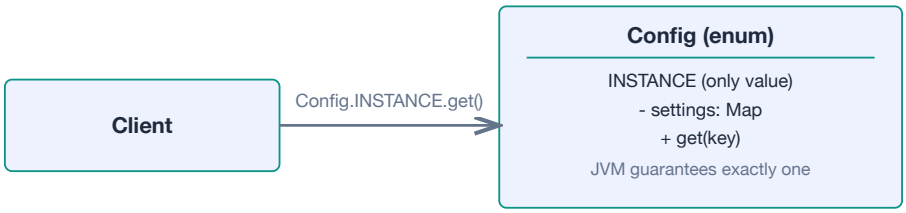

*Figure — every **Client** reaches the same, single **Config.INSTANCE**; no second instance can ever be created.*

JAVA EXAMPLE

```
// Preferred: enum singleton — thread-safe and concise for free
enum Config {
    INSTANCE;
    private final Map<String, String> settings = new HashMap<>();
    public String get(String key) { return settings.get(key); }
}
// usage: Config.INSTANCE.get("timeout")

// Lazy-init alternative: holder idiom (thread-safe, no locking needed)
class Database {
    private Database() {}
    private static class Holder { static final Database INSTANCE = new Database(); }
    public static Database getInstance() { return Holder.INSTANCE; }
}

// If you must lazily init with custom logic: double-checked locking
class Legacy {
    private static volatile Legacy instance;   // volatile is required here
    public static Legacy getInstance() {
        if (instance == null) {
            synchronized (Legacy.class) {
                if (instance == null) instance = new Legacy();
            }
        }
        return instance;
    }
}
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- Spring beans are singletons *by default scope* — but the key distinction is they're managed and **injected** by the container, never fetched through a static `getInstance()` call scattered through the codebase.
- `Runtime.getRuntime()` is the JDK's own classic (static-accessor style) singleton.

PITFALLS & CRITICISMS

Singleton is widely considered an **anti-pattern** in modern engineering practice, and for good reason:

- It introduces hidden global, mutable state that any code, anywhere, can read or change — the opposite of the encapsulation the rest of this book is trying to protect.
- It creates tight coupling: classes that call `Singleton.getInstance()` directly can't easily be tested with a fake, because the dependency isn't visible in the constructor or method signature.
- Classic (non-enum) implementations are a minefield of concurrency hazards — the double-checked locking idiom above is easy to get subtly wrong without `volatile`.
- It's genuinely hard to unit test: static state persists across test cases unless you're careful, causing flaky, order-dependent tests.

The generally accepted fix is **Dependency Injection**: let a container (Spring, Guice) own the single instance and hand it to whoever needs it through a constructor, rather than every class reaching out to a static accessor.

Staff-eng lens: this is the pattern most likely to be a trap question in a staff interview — the "correct" senior answer is usually not "how do I implement it correctly" but "why I'd avoid it." If you truly need exactly-one-instance semantics, get that from your DI container's default singleton scope and inject the dependency; that gives you the same guarantee with testability and explicit dependencies intact. Reserve a hand-written Singleton for narrow, low-level cases (e.g. a JVM-wide cache with no DI framework available) and say so explicitly.

Structural Patterns

Structural patterns are about **composing classes and objects into larger structures** while keeping things flexible. They don't worry about how an object gets created (that's creational) or how it behaves over time (that's behavioral) — they worry about the *shape* of relationships: who wraps whom, who holds a reference to whom, and how a tree of parts can be treated like a single whole. The common theme across all seven: use composition and well-placed interfaces so pieces can change independently without rippling through the whole codebase.

S Adapter

Intent (plain English): Convert the interface of a class into another interface that clients expect, so two incompatible things can work together.

Real-world analogy: A power plug adapter — your laptop charger has a US plug, the wall socket is European. You don't rewire the wall or the charger; you plug a small adapter in between.

PROBLEM IT SOLVES

You have a class with the exact behavior you need, but its method signatures don't match the interface your code already depends on. Rewriting either side is risky or impossible (e.g. it's a legacy class or a third-party library).

HOW IT WORKS

An **Adapter** class implements the interface the client expects (**Target**), and internally delegates calls to the incompatible class (the **Adaptee**). There are two flavors: an **object adapter** holds the adaptee via composition (works in Java, since Java has no multiple inheritance of classes) and a **class adapter** extends the adaptee directly (only works cleanly when your language allows multiple inheritance, or when the adaptee is itself an interface). Prefer the object adapter — it's more flexible and doesn't lock you into a subclass relationship.

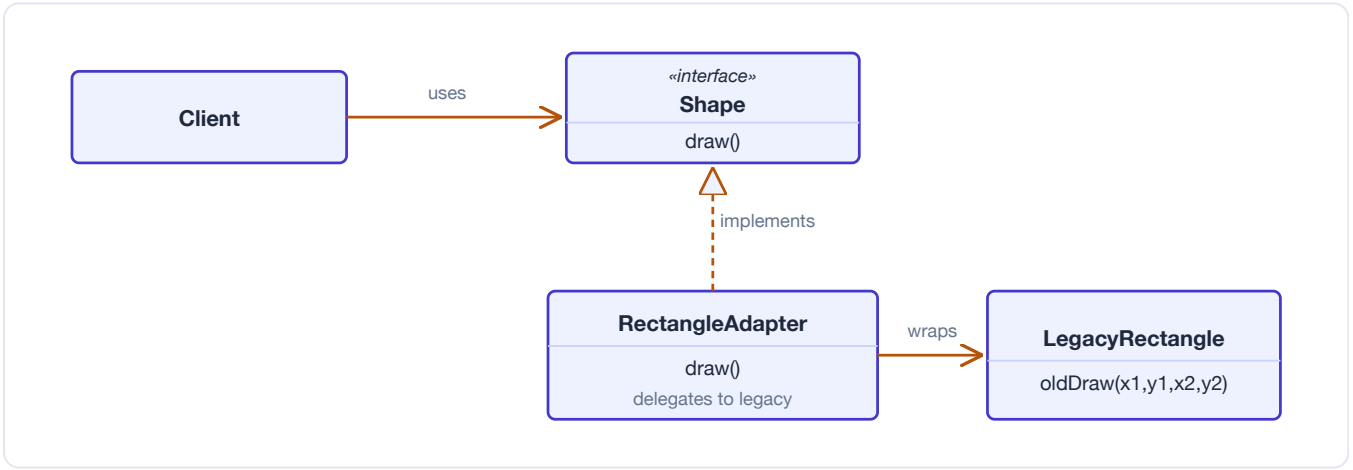

Figure — Object adapter: RectangleAdapter implements Shape and delegates to LegacyRectangle.

JAVA EXAMPLE

```
interface Shape { void draw(); }

class LegacyRectangle {                       // the "adaptee" — incompatible API
    void oldDraw(int x1, int y1, int x2, int y2) {
        System.out.printf("Rect (%d,%d)-(%d,%d)%n", x1, y1, x2, y2);
    }
}

class RectangleAdapter implements Shape {      // object adapter: wraps via composition
    private final LegacyRectangle legacy;
    private final int w, h;
    RectangleAdapter(LegacyRectangle legacy, int w, int h) {
        this.legacy = legacy; this.w = w; this.h = h;
    }
    @Override public void draw() { legacy.oldDraw(0, 0, w, h); }
}

Shape shape = new RectangleAdapter(new LegacyRectangle(), 100, 50);
shape.draw();               // client only ever talks to Shape
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `Arrays.asList()` adapts an array to the `List` interface.
- `Collections.list()` adapts an `Enumeration` to a `List`.
- `InputStreamReader`/`OutputStreamWriter` adapt byte streams to character streams.
- Spring MVC's `HandlerAdapter` lets `DispatcherServlet` invoke controllers written in different styles through one uniform call.

PITFALLS & CRITICISMS

Adapters can pile up when you're gluing together too many mismatched systems — that's often a sign the underlying design needs to change, not just be papered over. Class adapters (inheritance-based) tie you to one adaptee at compile time; object adapters are almost always the better default in Java.

Staff-eng lens: Reach for Adapter when integrating a third-party or legacy component you can't change. In an interview, the signal you want to give is: "I'd isolate the incompatibility behind a small adapter so the rest of the codebase depends only on my own interface" — that's also how you keep vendor lock-in low.

S Bridge

Intent (plain English): Decouple an abstraction from its implementation so the two can vary independently.

Real-world analogy: A TV remote control (abstraction) works with any brand of TV (implementation) as long as both speak the same signal protocol. You can swap the remote or the TV without touching the other.

PROBLEM IT SOLVES

If you model "shape types × rendering backends" purely with subclassing, you get a combinatorial explosion: `RedCircle`, `BlueCircle`, `RedSquare`, `BlueSquare`... every new shape or every new color doubles the class count.

HOW IT WORKS

Split the hierarchy in two: an **abstraction** hierarchy (e.g. `Shape`, `CircleShape`) and an **implementor** hierarchy (e.g. `DrawingAPI`, `SvgDrawingAPI`). The abstraction holds a *reference* to an implementor instead of extending it — that reference is the "bridge." Now shapes and drawing backends each grow their own hierarchy independently, and you combine them at object-construction time instead of at class-declaration time.

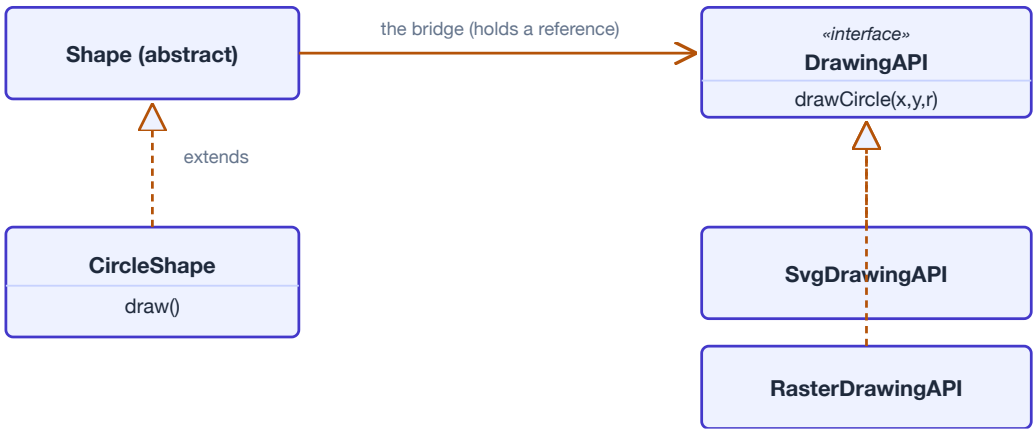

Figure — Two independent hierarchies (Shape, DrawingAPI) joined by a bridge reference, not inheritance.

JAVA EXAMPLE

```
interface DrawingAPI { void drawCircle(double x, double y, double r); }

class SvgDrawingAPI implements DrawingAPI {          // implementor #1
    public void drawCircle(double x, double y, double r) {
        System.out.printf("<circle cx='%.0f' cy='%.0f' r='%.0f'/>%n", x, y, r);
    }
}

class RasterDrawingAPI implements DrawingAPI {       // implementor #2
    public void drawCircle(double x, double y, double r) {
        System.out.printf("pixel-circle at (%.0f,%.0f) r=%.0f%n", x, y, r);
    }
}

abstract class Shape {
    protected final DrawingAPI api;                  // the "bridge" reference
    Shape(DrawingAPI api) { this.api = api; }
    abstract void draw();
}

class CircleShape extends Shape {
    private final double x, y, radius;
    CircleShape(double x, double y, double radius, DrawingAPI api) {
        super(api); this.x = x; this.y = y; this.radius = radius;
    }
    void draw() { api.drawCircle(x, y, radius); }
}
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- JDBC: your code programs against `java.sql.Connection`/`Driver` (abstraction) while vendor-specific driver JARs (implementors) plug in underneath.
- SLF4J: application code calls one logging API while Logback, Log4j2, or java.util.logging do the actual work as swappable implementors.

PITFALLS & CRITICISMS

Adds an extra layer of indirection up front for a problem you might not have yet — if you truly only ever need one implementation, Bridge is premature. It also gets confused with Adapter constantly: Bridge is designed in *up front* to let two hierarchies evolve together; Adapter is applied *after the fact* to reconcile something that already exists.

Staff-eng lens: Use Bridge when you can already see two dimensions of variation (e.g. shape type × renderer, notification type × channel) and subclassing would multiply. Don't reach for it just because "it might be flexible later" — that's speculative generality, a real cost in review time and cognitive load.

S Composite

Intent (plain English): Compose objects into tree structures to represent part-whole hierarchies, and let clients treat a single object and a whole group of objects the same way.

Real-world analogy: A file system: a folder can contain files or other folders, and "what's the total size?" works the same way whether you ask a single file or an entire folder tree.

PROBLEM IT SOLVES

Without Composite, client code has to constantly check "is this a leaf or a container?" and branch accordingly, which spreads type-checking logic everywhere a hierarchy is walked.

HOW IT WORKS

Define one interface (`FileSystemNode`) that both leaves (`File`) and containers (`Directory`) implement. A `Directory` holds a list of child `FileSystemNodes` (which may themselves be directories) and implements its operations by delegating to and combining its children's results. Client code calls the same method on the root and never needs to know how deep the tree goes.

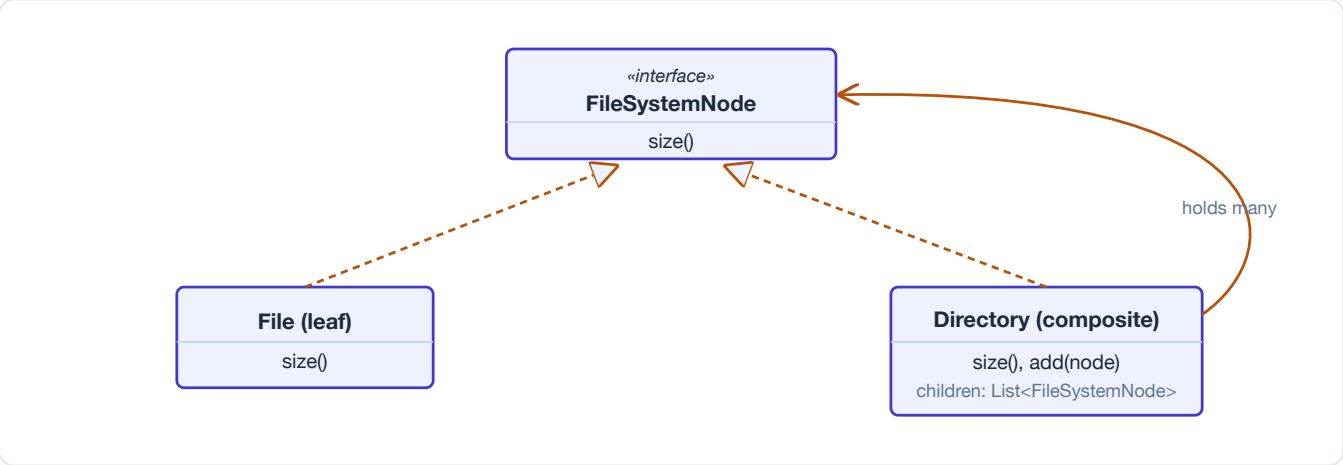

Figure — Directory holds a list of FileSystemNode children (files or more directories), all through one interface.

JAVA EXAMPLE

```
interface FileSystemNode { long size(); }

class FileLeaf implements FileSystemNode {        // leaf
    private final long bytes;
    FileLeaf(long bytes) { this.bytes = bytes; }
    public long size() { return bytes; }
}

class Directory implements FileSystemNode {       // composite
    private final List<FileSystemNode> children = new ArrayList<>();
    void add(FileSystemNode node) { children.add(node); }
    public long size() {
        return children.stream().mapToLong(FileSystemNode::size).sum();
    }
}

Directory root = new Directory();
root.add(new FileLeaf(1200));
Directory docs = new Directory();
docs.add(new FileLeaf(500));
root.add(docs);                                    // tree nests uniformly
System.out.println(root.size());                  // 1700 — no special-casing
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- Swing/AWT: every `Container` is itself a `Component`, so panels can hold panels holding buttons, and layout/paint code treats them uniformly.
- The DOM: every node (element, text, comment) implements `Node`, and elements can contain more nodes.
- `java.io.File` trees walked via `File.listFiles()` follow the same part-whole shape (though `File` itself isn't a strict GoF Composite class).

PITFALLS & CRITICISMS

Making leaves and composites share one interface sometimes forces leaf classes to implement operations that don't make sense for them (e.g. `add(child)` on a `File`), which either throws `UnsupportedOperationException` or silently no-ops — both are minor interface-purity violations you accept for the uniformity payoff.

Staff-eng lens: Great whenever you have a genuine recursive part-whole structure (org charts, UI trees, menu structures, permission scopes). Watch for stack depth / recursion limits on very deep trees, and think about whether traversal needs to be iterative for huge hierarchies.

S Decorator

Intent (plain English): Attach additional responsibilities to an object dynamically, without changing its class or affecting other instances.

Real-world analogy: Ordering coffee and adding toppings — milk, then sugar, then whipped cream. Each topping wraps the previous drink and adds its own cost and description, in whatever combination you like, decided at order time.

PROBLEM IT SOLVES

If you try to cover every combination of optional behaviors with subclasses (`CoffeeWithMilk`, `CoffeeWithMilkAndSugar`, ...), you get the same combinatorial explosion Bridge solves for a different axis — and it's all fixed at compile time, so you can't add a topping at runtime based on a user's order.

HOW IT WORKS

A **Decorator** implements the same interface as the object it wraps, holds a reference to that wrapped object, and forwards calls to it — adding its own behavior before or after. Because a decorator *is* a `Coffee` and also *holds* a `Coffee`, you can stack decorators arbitrarily deep, and the client code never notices — it just sees a `Coffee`. This is composition standing in for what would otherwise require many rigid subclasses, and unlike inheritance, the choice of what to add happens at runtime, not compile time.

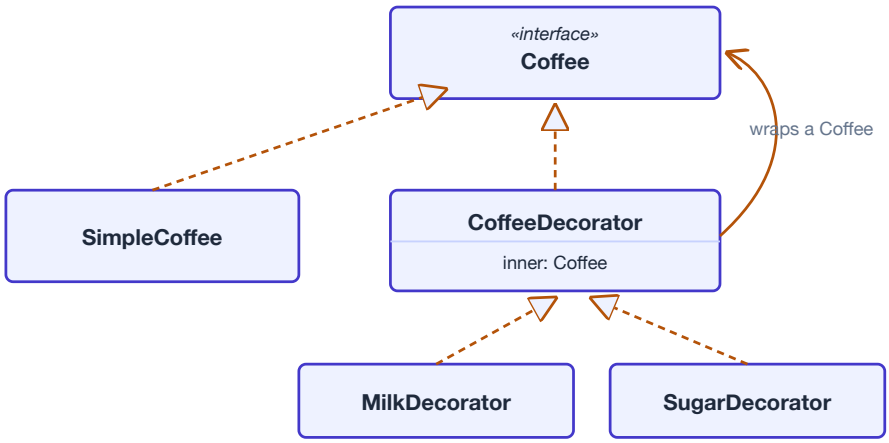

Figure — CoffeeDecorator implements Coffee and wraps another Coffee; concrete decorators stack on top.

JAVA EXAMPLE

```
interface Coffee { double cost(); String description(); }

class SimpleCoffee implements Coffee {
    public double cost() { return 2.0; }
    public String description() { return "Coffee"; }
}

abstract class CoffeeDecorator implements Coffee {
    protected final Coffee inner;
    CoffeeDecorator(Coffee inner) { this.inner = inner; }
}

class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee inner) { super(inner); }
    public double cost() { return inner.cost() + 0.5; }
    public String description() { return inner.description() + " + milk"; }
}

class SugarDecorator extends CoffeeDecorator {
    SugarDecorator(Coffee inner) { super(inner); }
    public double cost() { return inner.cost() + 0.2; }
    public String description() { return inner.description() + " + sugar"; }
}

Coffee order = new SugarDecorator(new MilkDecorator(new SimpleCoffee()));
System.out.println(order.description() + " = $" + order.cost());
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `java.io` streams are the textbook example: `new BufferedInputStream(new FileInputStream("f.txt"))` — each layer wraps the last and adds behavior (buffering, decompression, etc).
- `Collections.unmodifiableList(list)` and `Collections.synchronizedList(list)` wrap a `List` to add a restriction/behavior without changing its class.
- `HttpServletRequestWrapper` lets servlet filters wrap and augment a request before it reaches the next filter.

PITFALLS & CRITICISMS

Deep decorator stacks get hard to debug — a stack trace through five nested wrappers is not fun, and object identity/equality can get confusing (a decorated object isn't == the original, and might not `.equals()` it either depending on implementation). Compare to inheritance: subclassing adds behavior at compile time to a fixed type; Decorator adds it at runtime to a chosen instance — more flexible, but less discoverable by just reading the class hierarchy.

Staff-eng lens: This is a top interview pattern because `java.io` makes it concrete and memorable. Key point to articulate: decorator composition trades a little runtime indirection and debuggability for avoiding a subclass explosion, and it keeps the "core" class free of every possible optional feature.

S Facade

Intent (plain English): Provide a single, simple, unified interface to a set of interfaces in a complex subsystem.

Real-world analogy: Calling one customer-service phone number instead of trying to figure out which of twelve departments to call yourself — the facade routes your request internally.

PROBLEM IT SOLVES

A subsystem (video conversion, an ORM, a payment gateway) has many classes with intricate call order and setup requirements. Client code that talks directly to all of them becomes tightly coupled to internal details that can change.

HOW IT WORKS

Introduce one class that knows how to drive the subsystem correctly and exposes just a couple of high-level methods. The facade doesn't hide the subsystem completely (advanced callers can still use it directly) — it just gives everyone else a simpler front door. Unlike Adapter, a facade isn't reconciling incompatible interfaces; it's simplifying and aggregating already-compatible ones.

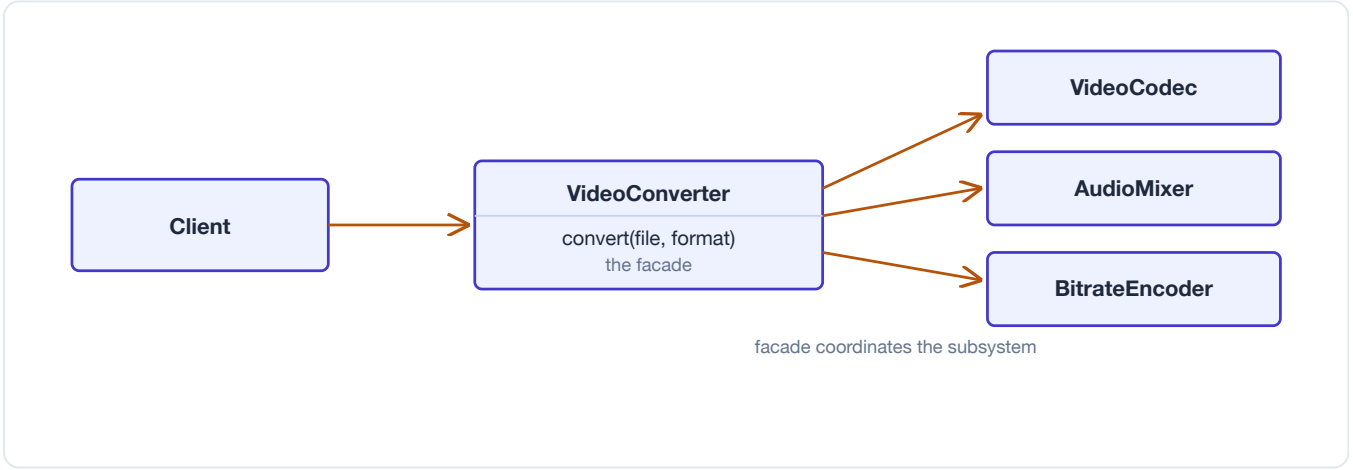

Figure — Client calls one facade method; the facade drives three subsystem classes in the right order.

JAVA EXAMPLE

```
class VideoCodec { void configure(String format) { /* ... */ } }
class AudioMixer { void normalize() { /* ... */ } }
class BitrateEncoder { void encode(int kbps) { /* ... */ } }

class VideoConverter {                            // the facade
    private final VideoCodec codec = new VideoCodec();
    private final AudioMixer mixer = new AudioMixer();
    private final BitrateEncoder encoder = new BitrateEncoder();

    String convert(String file, String format) {
        codec.configure(format);
        mixer.normalize();
        encoder.encode(1200);
        return file.replaceAll("\\.\\w+$", "." + format);
    }
}

VideoConverter converter = new VideoConverter();
String output = converter.convert("movie.mov", "mp4");  // one call, three subsystems
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- SLF4J is a facade over multiple logging backends (Logback, Log4j2, JUL).
- Spring's `JdbcTemplate` is a facade over raw JDBC (connections, statements, result sets, exception translation).
- `javax.faces` (JSF) provides a facade-like unified API over lower-level servlet/web mechanics.

PITFALLS & CRITICISMS

A facade can turn into a "god object" if you keep bolting on methods for every subsystem use case instead of keeping it thin. It can also hide performance-relevant details (e.g. batching, connection reuse) that power users need to bypass the facade for.

Staff-eng lens: Facade is one of the simplest, highest-ROI patterns — you'll use it constantly at the boundary of any subsystem or third-party integration. Interview tip: mention that a facade is about a simpler API surface, not about restricting access (that's closer to Proxy) or converting an incompatible one (that's Adapter).

S Flyweight

Intent (plain English): Use sharing to support huge numbers of fine-grained objects efficiently, by separating what can be shared from what can't.

Real-world analogy: A print shop doesn't cast a brand-new letter "A" stencil for every document — one "A" stencil (shared) gets stamped at many different positions on many different pages (the position is supplied each time, not stored in the stencil).

PROBLEM IT SOLVES

If you model a text document as one object per character, or a forest as one object per tree, memory usage explodes — most of that per-object data (font, glyph shape, tree species) is identical across thousands of instances.

HOW IT WORKS

Split an object's state into **intrinsic state** (shared, immutable, stored in the flyweight itself — e.g. the glyph shape and font) and **extrinsic state** (context-specific, passed in by the caller at the moment of use — e.g. the x/y position). A factory hands out shared flyweight instances from a pool keyed by intrinsic state, creating a new one only the first time a given combination is requested.

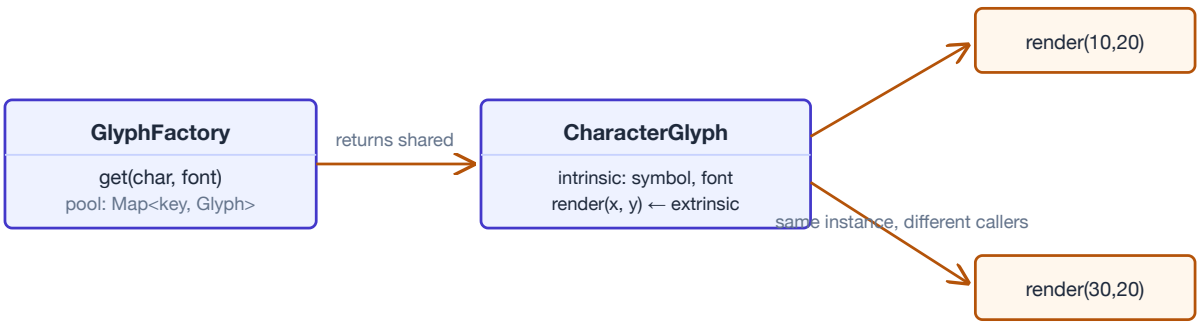

Figure — One shared CharacterGlyph instance is rendered at two different extrinsic positions.

JAVA EXAMPLE

```
class CharacterGlyph {                    // intrinsic state: shared, immutable
    private final char symbol;
    private final String font;
    CharacterGlyph(char symbol, String font) { this.symbol = symbol; this.font = font; }
    void render(int x, int y) {             // extrinsic state passed in per call
        System.out.printf("'%c' [%s] at (%d,%d)%n", symbol, font, x, y);
    }
}

class GlyphFactory {
    private static final Map<String, CharacterGlyph> pool = new HashMap<>();
    static CharacterGlyph get(char c, String font) {
        return pool.computeIfAbsent(c + font, k -> new CharacterGlyph(c, font));
    }
}

GlyphFactory.get('A', "Arial").render(10, 20);
GlyphFactory.get('A', "Arial").render(30, 20);  // same shared instance reused
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `Integer.valueOf()` caches boxed integers in the range -128..127 — one shared `Integer` object per value instead of a new object every time.
- The `String` intern pool shares one copy of each unique literal/interned string.
- `Boolean.valueOf()` always returns one of two shared singletons (TRUE/FALSE).

PITFALLS & CRITICISMS

Only worth it when object count is genuinely huge and memory pressure is real — otherwise it's needless complexity for a problem you don't have. Sharing mutable state by accident is the classic bug: if intrinsic state isn't actually immutable, one caller's mutation leaks into every other holder of that shared flyweight.

Staff-eng lens: The `Integer` cache example is a favorite interview trap: it's exactly why `Integer a = 100; Integer b = 100; a == b` is `true` but at 200 it's `false` — that's Flyweight leaking into observable behavior via reference equality. Good staff-level insight: know when NOT to use it — modern JVMs and escape analysis make manual object pooling less necessary than it used to be for small objects.

S Proxy

Intent (plain English): Provide a stand-in object that controls access to another (usually expensive, remote, or sensitive) object.

Real-world analogy: A credit card is a proxy for the cash sitting in your bank account — it controls, authorizes, and records access to the real thing without you handling the actual money directly.

PROBLEM IT SOLVES

Sometimes you can't or shouldn't create/access the real object immediately: it's expensive to construct (a large image), lives on another machine (a remote service), or needs access checks (only admins can call it). Calling code shouldn't have to know any of that.

HOW IT WORKS

The proxy implements the same interface as the real object and holds a reference to it (created lazily, remotely, or conditionally). Common flavors: **virtual proxy** (defers expensive creation until first use), **protection proxy** (checks permissions before delegating), **remote proxy** (represents an object living in another process/machine), and **caching proxy** (returns a cached result instead of recomputing). The client code is unaware which one it's talking to.

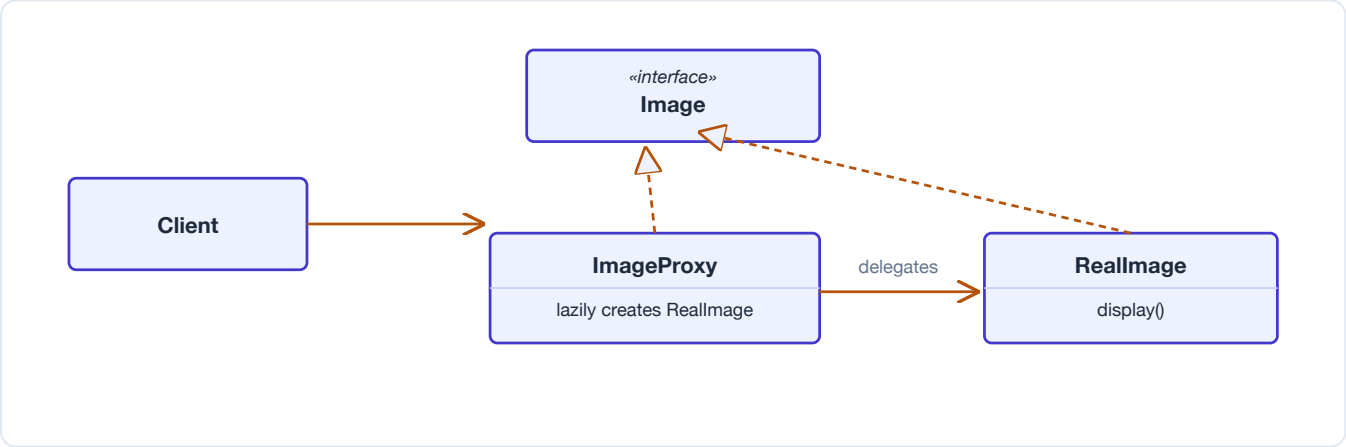

Figure — Client only sees Image; ImageProxy defers creating the expensive RealImage until display() is first called.

JAVA EXAMPLE

```
interface Image { void display(); }

class RealImage implements Image {           // expensive to create
    private final String path;
    RealImage(String path) { this.path = path; loadFromDisk(); }
    private void loadFromDisk() { System.out.println("Loading " + path); }
    public void display() { System.out.println("Displaying " + path); }
}

class ImageProxy implements Image {          // virtual proxy: lazy init
    private final String path;
    private RealImage real;
    ImageProxy(String path) { this.path = path; }
    public void display() {
        if (real == null) real = new RealImage(path);   // load on first use only
        real.display();
    }
}

Image image = new ImageProxy("diagram.png");  // cheap — nothing loaded yet
image.display();                              // now it loads and displays
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- Spring AOP proxies power `@Transactional`, `@Cacheable`, `@Async`: a JDK dynamic proxy (`java.lang.reflect.Proxy`) wraps interfaces, and CGLIB subclasses concrete classes at runtime — you'll literally see class names like `Foo$$EnhancerBySpringCGLIB$$abcd1234` in stack traces.
- Hibernate lazy-loaded entities are proxies that fetch data from the database only when a field is first accessed.
- RMI stubs are remote proxies standing in for an object that actually lives on a different JVM/machine.

PITFALLS & CRITICISMS

Dynamic proxies (Spring, Hibernate) can produce confusing bugs: calling an `@Transactional` method from *within the same class* bypasses the proxy entirely (self-invocation problem), and lazy entity proxies thrown around after a session closes cause the infamous `LazyInitializationException`. Proxies also add a layer that's invisible in the source but very visible in stack traces and debuggers.

> **Staff-eng lens:** This is one of the highest-value patterns to know cold for a staff interview because it explains real framework "magic" — why `@Transactional` silently does nothing when self-invoked, why CGLIB needs a no-arg constructor and non-final classes/methods. Being able to explain proxy mechanics is a strong signal of Spring/Hibernate internals depth.

ADAPTER VS DECORATOR VS PROXY VS FACADE — THEY ALL "WRAP," BUT FOR DIFFERENT REASONS

Pattern	Same interface as wrapped object?	Purpose	One-line test
Adapter	No — implements a *different* target interface	Convert an incompatible interface into one the client expects	"These two don't speak the same language."
Decorator	Yes — same interface, stackable	Add behavior dynamically, any combination, at runtime	"I want to add features without new subclasses."
Proxy	Yes — same interface, usually one layer	Control access (lazy load, permission check, remote call, cache)	"The real object needs a gatekeeper."
Facade	No — new, simpler interface over many classes	Simplify/aggregate a whole subsystem behind one entry point	"There are ten classes here; give me one method."

Behavioral Patterns

Behavioral patterns are about **who talks to whom, and how**. Creational patterns handle object birth, structural patterns handle object shape, and behavioral patterns handle object *conversation* — how responsibilities are distributed among objects and how they collaborate without turning into a tangled mess of if/else chains and direct references. The common theme across this family: instead of hard-wiring "Object A calls Object B calls Object C," you introduce a small abstraction (a handler chain, a command object, a mediator, an iterator) that keeps the collaborators loosely coupled and lets the collaboration itself vary independently. For a staff engineer, this family is where interviewers probe whether you can spot "shotgun surgery" and coupling smells and reach for the right amount of indirection — not too little, not too much.

B Chain of Responsibility

Intent (plain English): Give a request a chance to be handled by any one of several handlers, by lining the handlers up and passing the request along the line until someone deals with it.

Real-world analogy: A support ticket that starts with tier-1 support. If tier-1 can't solve it, it escalates to tier-2, then tier-3, then an engineer. Nobody at the bottom needs to know who eventually solves it — they just pass it up if they can't.

PROBLEM IT SOLVES

You have a request that could be handled by one of several objects, but you don't know which one in advance, and you don't want the sender to hold a giant if/else or switch statement listing every possible handler. You want to add, remove, or reorder handlers without touching the sender's code.

HOW IT WORKS

Each handler implements a common interface (or extends a common abstract class) with a **handle(request)** method and a reference to the **next** handler in the chain. A handler either processes the request itself, or forwards it to `next`. The client only ever talks to the first handler in the chain — it has no idea how many links exist or which one will actually act.

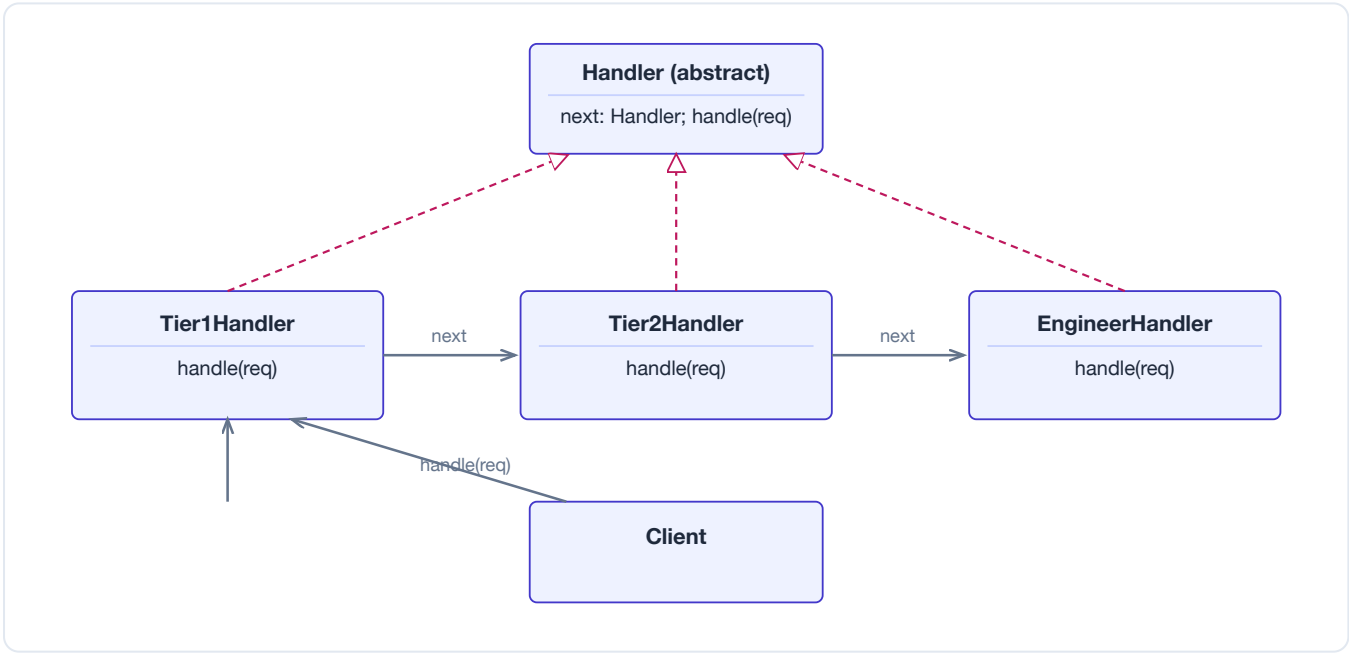

Figure — a chain of handlers; the client only knows about the first link.

JAVA EXAMPLE

```
abstract class ApprovalHandler {
    protected ApprovalHandler next;
    ApprovalHandler setNext(ApprovalHandler next) { this.next = next; return next; }
    abstract void handle(Expense expense);
}

class ManagerApproval extends ApprovalHandler {
    void handle(Expense expense) {
        if (expense.amount() <= 1000) System.out.println("Manager approved");
        else if (next != null) next.handle(expense);
        else System.out.println("No one could approve this");
    }
}

class DirectorApproval extends ApprovalHandler {
    void handle(Expense expense) {
        if (expense.amount() <= 10000) System.out.println("Director approved");
        else if (next != null) next.handle(expense);
    }
}

// wiring: manager.setNext(director).setNext(vp);
// client only calls manager.handle(expense);
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `javax.servlet.Filter` chains — each filter can act and/or call `chain.doFilter(...)` to pass along.
- Spring Security's filter chain — a literal ordered list of security filters, each deciding to handle or delegate.
- `java.util.logging.Logger` handlers, which propagate log records up parent loggers.
- Netty's `ChannelPipeline`, where each `ChannelHandler` can process or forward an event.

PITFALLS & CRITICISMS

If nobody in the chain handles the request, it silently disappears unless you build in a "no handler found" fallback. Long chains can be hard to debug — you must inspect the wiring code to know the order. It's also easy to accidentally build a chain that's really just a disguised if/else ladder split across many files, adding indirection without real benefit.

Staff-eng lens: reach for this when the set of possible handlers is genuinely open-ended or configurable (plugins, filters, middleware). Don't use it for a fixed, small set of cases that will never grow — a simple switch statement is more debuggable. In interviews, this pattern is a good answer whenever someone asks "how would you build a pluggable request-processing pipeline?"

B Command

Intent (plain English): Turn a request into a standalone object so you can pass it around, queue it, log it, or undo it — instead of calling a method directly.

Real-world analogy: A restaurant order ticket. The waiter (invoker) doesn't cook — they just write down "table 4: medium steak" and hand the ticket to the kitchen (receiver). The ticket itself is an object that can be queued, handed to any cook, or even voided.

PROBLEM IT SOLVES

You want to decouple the thing that triggers an action (a button, a menu item, a scheduler) from the thing that knows how to perform it, so the trigger doesn't need to know the receiver's API. You also want to queue, log, or undo actions — which is hard if "doing the action" is just a bare method call.

HOW IT WORKS

A **Command** interface declares `execute()` (and often `undo()`). Each concrete command wraps a receiver and the specific action to invoke on it. An **Invoker** (like a remote-control button) holds a command and calls `execute()` without knowing what it actually does. Because commands are objects, you can stack them for undo/redo, put them in a queue, or serialize them.

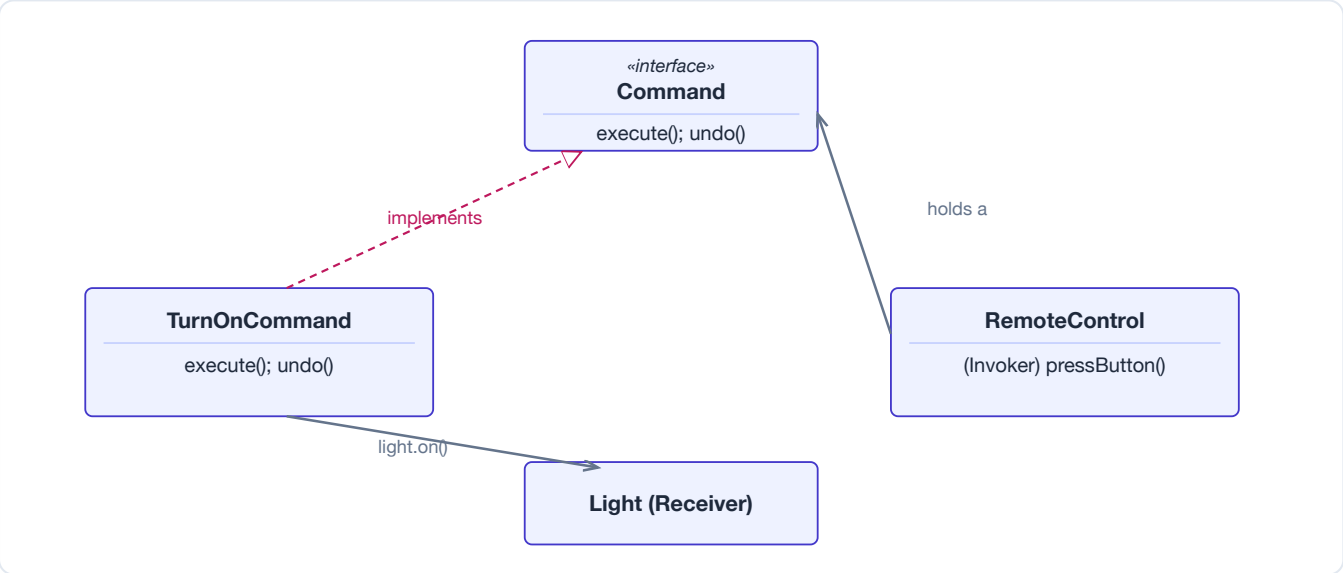

Figure — the invoker holds a Command; the command calls back into the receiver.

JAVA EXAMPLE

```
interface Command {
    void execute();
    void undo();
}

class Light {
    void on()  { System.out.println("Light ON"); }
    void off() { System.out.println("Light OFF"); }
}

class TurnOnCommand implements Command {
    private final Light light;
    TurnOnCommand(Light light) { this.light = light; }
    public void execute() { light.on(); }
    public void undo()    { light.off(); }
}

class RemoteControl {
    private final Deque<Command> history = new ArrayDeque<>();
    void press(Command command) { command.execute(); history.push(command); }
    void pressUndo() { if (!history.isEmpty()) history.pop().undo(); }
}
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `java.lang.Runnable` and `Callable<V>` — a unit of work as an object, handed to an `ExecutorService`.
- `javax.swing.Action` — a command bound to menu items, buttons, and keystrokes at once.
- Undo/redo stacks in editors (text editors, IDEs) — each edit is a command with an inverse.
- Job/message queues (e.g. task objects submitted to a queue for later execution).

PITFALLS & CRITICISMS

Every trivial action becomes a class, which can explode your codebase with tiny command classes for simple UIs. Implementing a correct `undo()` is often harder than it looks (you may need to snapshot state rather than just "do the opposite action") — that's where **Memento** often teams up with Command.

Staff-eng lens: use Command when you actually need queuing, logging, undo, or decoupled triggering (macros, schedulers, transactional operations). If you just need to pass a behavior once, a lambda/`Runnable` is enough — don't build a class hierarchy for it. Interviewers like to see you recognize when `Runnable`/`Callable` already *is* Command in disguise.

B Interpreter

Intent (plain English): For a small language you've invented, represent each grammar rule as a class, and give it a method that evaluates ("interprets") a sentence written in that language.

Real-world analogy: Reading a simple recipe notation like "2 eggs + 1 cup flour". Each part of the notation (a quantity, an ingredient, the "+" operator) has an obvious meaning, and you interpret the whole sentence by interpreting its parts and combining them.

PROBLEM IT SOLVES

You have a simple, recurring language or notation (a filter expression, a small formula, a rule syntax) and you want a clean, object-oriented way to represent and evaluate sentences in it, rather than writing a hand-rolled string parser tangled with evaluation logic.

HOW IT WORKS

Every grammar rule becomes an **Expression** type with an `interpret()` method. Terminal rules (like a literal number) interpret themselves directly. Non-terminal rules (like "add") hold references to sub-expressions and interpret by combining their children's results. The result is a tree of small objects that mirrors the grammar — walking the tree "runs" the sentence.

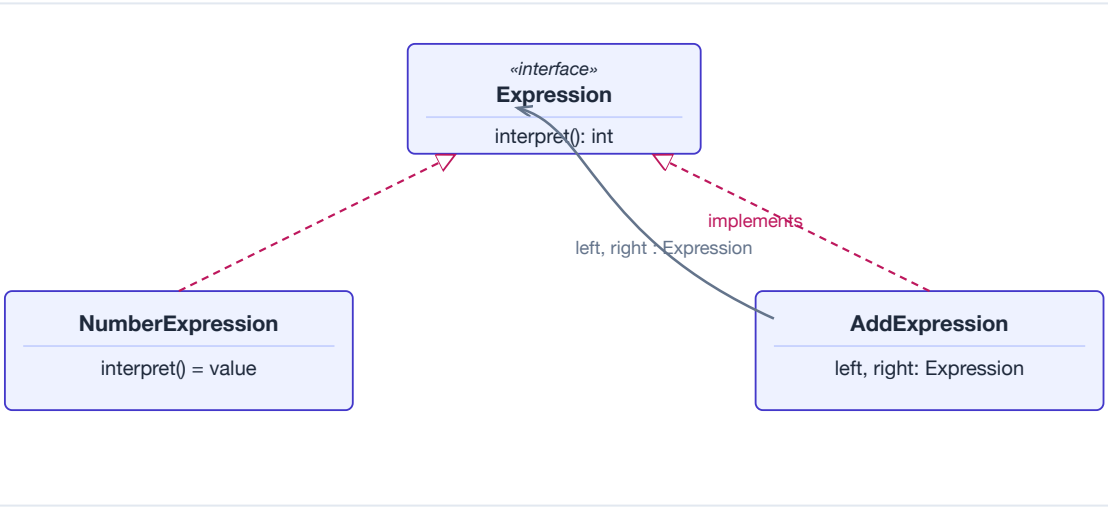

Figure — a composite expression tree; each node knows how to interpret itself.

JAVA EXAMPLE

```
interface Expression {
    int interpret();
}

record NumberExpression(int value) implements Expression {
    public int interpret() { return value; }
}

record AddExpression(Expression left, Expression right) implements Expression {
    public int interpret() { return left.interpret() + right.interpret(); }
}

// (2 + 3) modeled as a tree:
Expression twoPlusThree = new AddExpression(new NumberExpression(2), new
NumberExpression(3));
System.out.println(twoPlusThree.interpret()); // 5
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `java.util.regex.Pattern` — a regular expression is compiled into an internal tree of matching nodes and "interpreted" against input.
- Spring Expression Language (SpEL) — parses and evaluates expressions like `#{user.name}` in configuration and annotations.
- `java.text.Format` subclasses (e.g. `MessageFormat`) interpret a small pattern language for formatting.

PITFALLS & CRITICISMS

This pattern gets unwieldy fast for anything beyond a toy grammar — every new grammar rule is a new class, and parsing plus precedence handling by hand becomes a maintenance burden. For any real language, use a proper parser generator (e.g. **ANTLR**) or an existing expression library rather than hand-rolling the classic GoF tree.

> **Staff-eng lens:** this is the pattern you can namedrop and then immediately deflect — "I know the classic GoF Interpreter, but in practice I'd reach for ANTLR or an existing expression evaluator rather than roll my own tree of classes." Showing that judgment call is more impressive than reciting the pattern.

B Iterator

Intent (plain English): Let callers walk through the elements of a collection one at a time, without exposing how that collection is actually stored internally.

Real-world analogy: A TV remote's next/previous buttons. You press "next channel" without knowing whether channels are stored as a list, a linked structure, or fetched live from a satellite — the remote just knows how to get you to the next one.

PROBLEM IT SOLVES

You want to traverse different kinds of collections (arrays, linked lists, trees, custom structures) using the same client code, and you don't want the traversal logic to leak the collection's internal representation, or force it to expose a raw array/index.

HOW IT WORKS

The collection implements **Iterable<T>**, exposing an `iterator()` factory method. The returned **Iterator<T>** object tracks traversal position itself and exposes `hasNext()` / `next()`. The client only ever calls those two methods, so the same loop code works no matter what's underneath.

Figure — collection and iterator are separate roles, both implemented by concrete classes.

JAVA EXAMPLE

```
class NameCollection implements Iterable<String> {
    private final String[] names;
    NameCollection(String... names) { this.names = names; }

    public Iterator<String> iterator() {
        return new Iterator<>() {
            private int index = 0;
            public boolean hasNext() { return index < names.length; }
            public String next() { return names[index++]; }
        };
    }
}

// client code never sees the array:
for (String name : new NameCollection("Ada", "Grace", "Linus")) {
    System.out.println(name);
}
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `java.util.Iterator` is the canonical textbook example of this pattern — it superseded the older, more limited `Enumeration` interface.
- Every `Collection` (`ArrayList`, `HashSet`, `LinkedList`, ...) implements `Iterable` and hands back an `Iterator`.
- `java.util.Scanner` behaves as an iterator-like cursor over tokens in an input stream.

PITFALLS & CRITICISMS

A naive iterator can break (throw `ConcurrentModificationException`) if the underlying collection is mutated during iteration — a subtlety worth knowing cold for an interview. Custom iterators are also easy to get wrong around edge cases like empty collections or calling `next()` without checking `hasNext()` first.

Staff-eng lens: you'll rarely write a custom iterator from scratch — the JDK's is almost always enough. This pattern is really interview fodder for demonstrating you understand encapsulation of internal structure, and for discussing fail-fast iteration and concurrent modification pitfalls in real production incidents.

B Mediator

Intent (plain English): Instead of letting a group of objects talk directly to each other (and know about each other), route all their communication through one central object.

Real-world analogy: An air-traffic control tower. Pilots don't radio each other directly to negotiate who lands first — every plane talks to the tower, and the tower coordinates all of them. Remove the tower and add ten more planes, and direct plane-to-plane coordination becomes chaos.

PROBLEM IT SOLVES

When many objects reference each other directly to collaborate, you end up with a dense web of dependencies — a change to one object risks breaking several others, and the interaction logic gets duplicated across every object's code. You want to centralize "who talks to whom" so the participants stay simple and decoupled.

HOW IT WORKS

Participants (e.g. **User** objects in a chat room) hold a reference only to the **Mediator**, never to each other. When a participant wants to communicate, it tells the mediator; the mediator decides how to route or broadcast that to the other participants. Add a new participant type, and existing ones don't need to change at all.

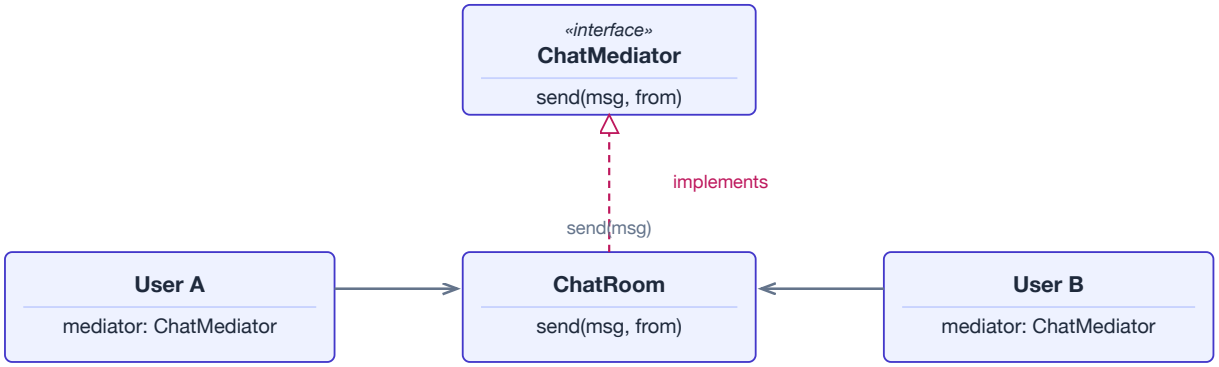

Figure — users only know the mediator; the mediator routes messages between them.

JAVA EXAMPLE

```
interface ChatMediator {
    void send(String message, User from);
    void register(User user);
}

class ChatRoom implements ChatMediator {
    private final List<User> users = new ArrayList<>();
    public void register(User user) { users.add(user); }
    public void send(String message, User from) {
        for (User u : users)
            if (u != from) u.receive(message, from);
    }
}

class User {
    private final String name;
    private final ChatMediator mediator;
    User(String name, ChatMediator mediator) {
        this.name = name; this.mediator = mediator; mediator.register(this);
    }
    void say(String message) { mediator.send(message, this); }
    void receive(String message, User from) { System.out.println(name + " got: " + message);
}
}
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `java.util.concurrent.Executor` / `ExecutorService` — code submitting a task never talks to the worker thread directly; the executor mediates.
- Spring's `ApplicationEventPublisher` — publishers and listeners never reference each other; the event bus mediates.
- General message buses / brokers (e.g. an in-process event bus) that decouple producers from consumers.

PITFALLS & CRITICISMS

All the complexity you removed from the participants has to go *somewhere* — the mediator itself can become a bloated "god object" that knows too much about everyone. It's a classic case of moving coupling rather than eliminating it; the win is worthwhile only when the participant-to-participant web was worse.

Staff-eng lens: great answer when asked "how would you decouple these five UI widgets / services that all reference each other?" But flag the tradeoff explicitly: a mediator concentrates risk and complexity in one place, so it needs its own tests and clear boundaries — don't let it silently grow into a monolith that does all the real work.

B Memento

Intent (plain English): Capture an object's internal state so you can restore it later, without exposing or breaking that object's encapsulation.

Real-world analogy: A save-game checkpoint. The game state gets written to a save file at a point in time; later, you can load that file to jump back to exactly that point — without the save file exposing or letting you tamper with the game engine's internals directly.

PROBLEM IT SOLVES

You want undo/rollback for an object's state, but grabbing that state naively (e.g. exposing all its private fields via getters) breaks encapsulation and couples the "history keeper" to the object's internal structure — meaning any internal refactor of the object breaks the undo feature too.

HOW IT WORKS

The **Originator** (the object whose state matters) creates a **Memento** — an opaque, immutable snapshot of its own state — via a `save()` method, and can later restore itself from one via `restore(memento)`. A separate **Caretaker** just stores mementos (often in a stack) for later use, but never looks inside them.

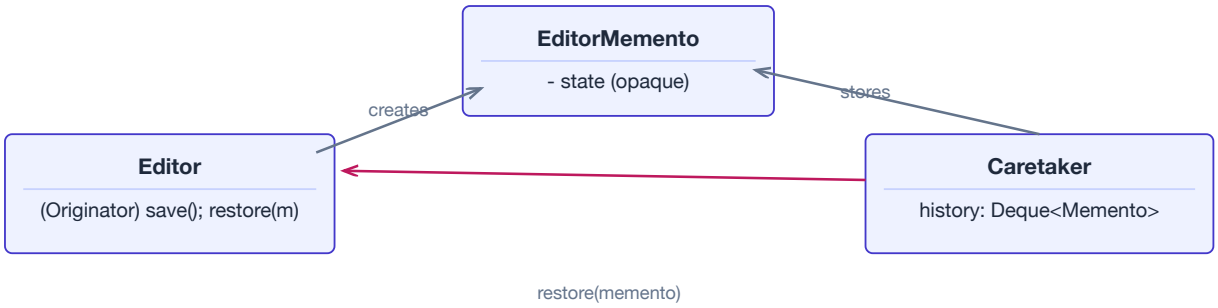

Figure — Editor creates opaque mementos; Caretaker stacks them without peeking inside.

JAVA EXAMPLE

```
final class EditorMemento {
    private final String text; // package-private: only Editor can read this
    EditorMemento(String text) { this.text = text; }
    String getText() { return text; }
}

class Editor {
    private String text = "";
    void type(String more) { text += more; }
    EditorMemento save() { return new EditorMemento(text); }
    void restore(EditorMemento m) { this.text = m.getText(); }
}

class Caretaker {
    private final Deque<EditorMemento> history = new ArrayDeque<>();
    void backup(Editor editor) { history.push(editor.save()); }
    void undo(Editor editor) { if (!history.isEmpty()) editor.restore(history.pop()); }
}
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- Undo/redo stacks in text editors and IDEs (often paired with **Command** — each command stores the memento needed to reverse itself).
- `java.io.Serializable` object graphs used purely as state snapshots (serialize now, deserialize later to restore).
- Database/transaction rollback: a "before" snapshot of a row or aggregate that gets restored if a transaction aborts.

PITFALLS & CRITICISMS

Snapshotting large or deeply-nested state can be expensive in memory and time if you save too often or keep too long a history. Keeping many mementos around (unbounded undo history) can quietly become a memory leak if the caretaker never trims old entries.

Staff-eng lens: the key encapsulation trick to call out in an interview is that the memento is opaque to the caretaker — only the originator can read its own snapshot. Worth mentioning the memory/perf tradeoff of full-state snapshots vs. storing a smaller diff/delta when state is large, which is exactly the kind of pragmatic follow-up a staff interview probes for.

B Observer

Intent (plain English): when one object changes, automatically tell everyone who cares — without that object needing to know who they are.

Real-world analogy: subscribing to a YouTube channel. The creator (the **Subject**) just publishes videos; they don't personally know each subscriber. Subscribers (**Observers**) opt in, get notified automatically, and can unsubscribe any time — the creator's code never changes.

PROBLEM IT SOLVES

Several objects need to react whenever another object's state changes, but you don't want that object hard-coding calls to every specific reactor. Hard-coding creates tight coupling — every time you add a new "reactor" you'd have to edit the source object.

HOW IT WORKS

The **Subject** keeps a list of **Observer** references and exposes `subscribe()`/`unsubscribe()`. When its state changes, it loops over the list and calls `update()` on each one. Because every observer implements the same small interface, the subject only ever depends on that interface — never on concrete observer classes. New observer types can be added with zero changes to the subject.

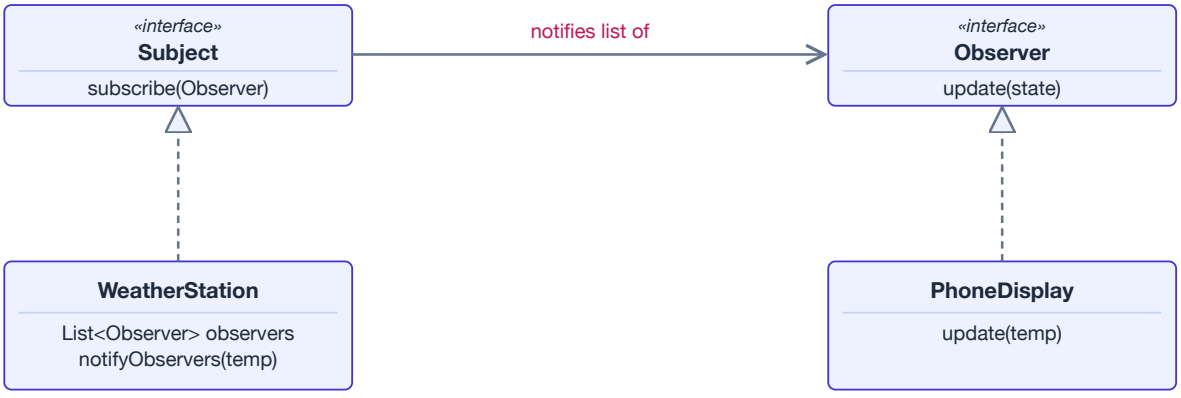

Figure — Subject broadcasts to any number of Observers through one shared interface.

JAVA EXAMPLE

```
interface Observer { void update(double temp); }

class WeatherStation {
    private final List<Observer> observers = new ArrayList<>();
    void subscribe(Observer o) { observers.add(o); }
    void unsubscribe(Observer o) { observers.remove(o); }
    void setTemperature(double temp) {
        for (Observer o : observers) o.update(temp); // notify all
    }
}

class PhoneDisplay implements Observer {
    public void update(double temp) {
        System.out.println("Phone shows: " + temp + "C");
    }
}

var station = new WeatherStation();
station.subscribe(new PhoneDisplay());
station.setTemperature(28.5); // PhoneDisplay is notified automatically
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `java.util.EventListener` family and Swing listeners (`ActionListener`, `ItemListener`).
- Spring's `ApplicationListener` interface and the `@EventListener` annotation.
- RxJava / Project Reactor — `Flux`/`Mono` subscribers are Observers on a reactive stream.
- `java.beans.PropertyChangeListener` + `PropertyChangeSupport`.
- Historical note: the original `java.util.Observer`/`Observable` classes were **deprecated in Java 9** — `Observable` wasn't an interface, wasn't thread-safe, and couldn't be composed. Use the JDK listener idiom or a reactive library instead.

PITFALLS & CRITICISMS

Notification order isn't guaranteed. Forgetting to `unsubscribe()` is a classic memory leak (the subject keeps a strong reference to every observer). Synchronous notification can block the subject if an observer is slow, and long observer chains can trigger surprising cascades of updates that are hard to trace while debugging.

Staff-eng lens: Observer is the in-process ancestor of pub/sub and reactive streams. Recognize it as the backbone of event-driven design — and know when to graduate from an in-process Observer to a real message broker (Kafka, SNS/SQS) once you need decoupling across processes/services rather than within one object graph.

B State

Intent (plain English): let an object change its behavior when its internal state changes, so it behaves as if it switched classes on the fly.

Real-world analogy: a vending machine behaves differently depending on whether it has *no coin*, *has a coin*, or is *dispensing*. Same buttons, same machine — but pressing "select" does something different in each state.

PROBLEM IT SOLVES

An object's behavior depends on a status flag, so every method ends up with `if (state == X) ... else if (state == Y) ...`. Add one new state and you must find and edit every one of those blocks — easy to miss one.

HOW IT WORKS

Pull each state's behavior into its own class implementing a common **State** interface. The **Context** object holds a reference to its *current* State object and simply delegates to it. The state object itself decides when to transition — it calls `context.setState(next)` — so the context's own code never needs a switch statement; its behavior "changes class" purely by swapping which State object it's holding.

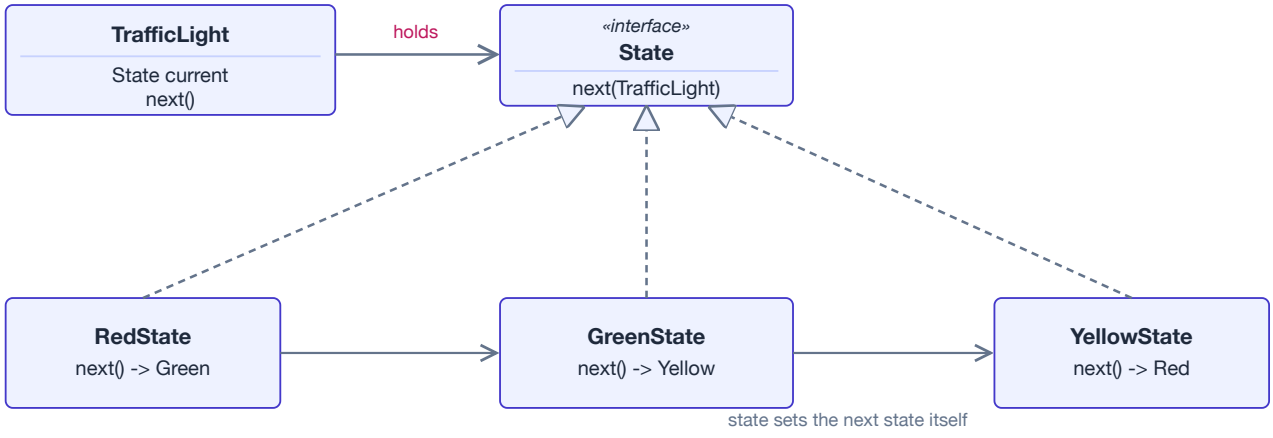

Figure — Context delegates to whichever State it currently holds; each State decides the next transition.

JAVA EXAMPLE

```
interface State { State next(); }

class RedState implements State {
    public State next() { System.out.println("Red -> Green"); return new GreenState(); }
}
class GreenState implements State {
    public State next() { System.out.println("Green -> Yellow"); return new YellowState(); }
}
class YellowState implements State {
    public State next() { System.out.println("Yellow -> Red"); return new RedState(); }
}

class TrafficLight {
    private State current = new RedState();
    void advance() { current = current.next(); } // delegate; behavior "changes class"
}
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- Order / workflow state machines (e.g. `PENDING -> PAID -> SHIPPED -> DELIVERED`) in e-commerce systems.
- Spring State Machine (`spring-statemachine`) — a full library built around this pattern.
- TCP connection states (`LISTEN`, `ESTABLISHED`, `CLOSED`) in networking stacks conceptually follow the same shape.

PITFALLS & CRITICISMS

Each state becomes its own class, which is overkill for two or three trivial states — a plain enum with a switch may read more clearly at small scale. If states carry no per-instance data, share single instances (or use enum constants implementing the interface) instead of allocating a new object per transition. Spreading transition logic across many small classes can also make the overall flow harder to see at a glance versus one table/diagram.

Staff-eng lens: the classic interview trio — **Strategy** vs **State** vs **Template Method** — share the "delegate to an interface" shape but differ in *who* controls change: Strategy's algorithm is picked once by the client and stays fixed; State transitions itself, driven by the object's own logic, invisible to the client; Template Method uses inheritance, not composition, and the skeleton — not the behavior object — is fixed. Reach for State only when there really is a meaningful number of states with distinct behavior, not just a boolean flag.

B Strategy

Intent (plain English): define a family of interchangeable algorithms, and let the client swap between them without touching the code that uses them.

Real-world analogy: choosing a route in a maps app. Driving, walking, and transit are different algorithms for the same "get me there" job — you switch modes without changing how you tap "Go".

PROBLEM IT SOLVES

You need several interchangeable variants of an algorithm (sort order, payment method, compression scheme) and don't want an ever-growing if/else chain selecting behavior, or a tangle of subclasses for every combination of behaviors.

HOW IT WORKS

Define one interface for the algorithm's signature. Put each variant behind its own implementation of that interface. The **Context** is configured with (or simply receives) whichever **Strategy** instance it should use, and calls it only through the interface. Swapping behavior is just swapping which object (or, in modern Java, which *lambda*) is plugged in — no inheritance required.

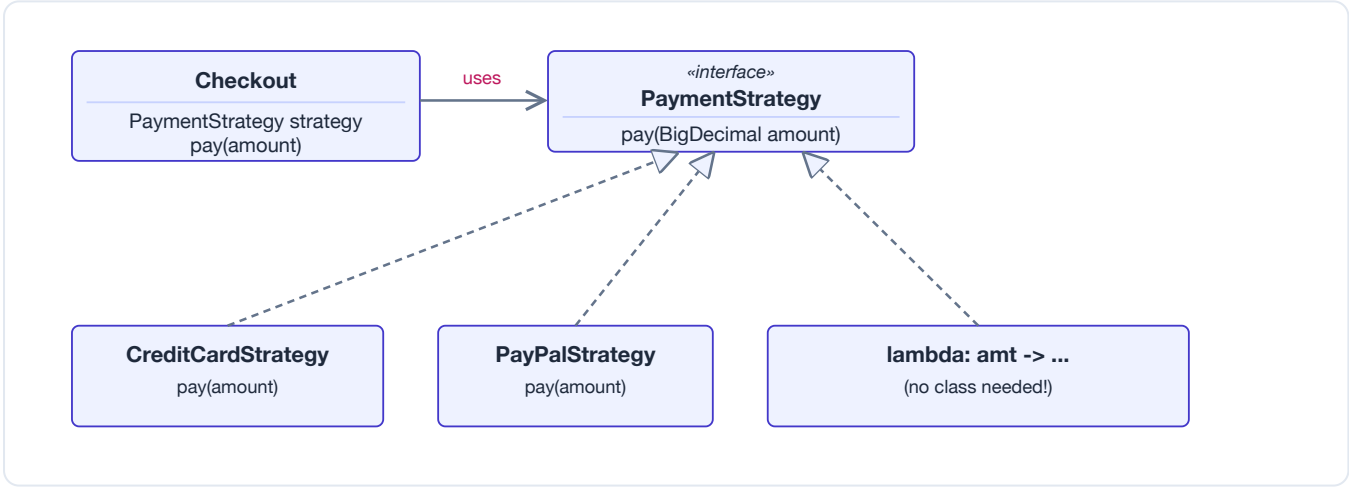

Figure — Context is wired to one interchangeable Strategy; a lambda can play the same role as a class.

JAVA EXAMPLE

```
interface PaymentStrategy { void pay(BigDecimal amount); }

class Checkout {
    private final PaymentStrategy strategy;
    Checkout(PaymentStrategy strategy) { this.strategy = strategy; }
    void completeOrder(BigDecimal total) { strategy.pay(total); }
}

// Classic style: a concrete class per algorithm
class PayPalStrategy implements PaymentStrategy {
    public void pay(BigDecimal amount) { System.out.println("Paid via PayPal: " + amount); }
}

// Modern style: the strategy is just a lambda
Checkout order = new Checkout(amt -> System.out.println("Paid by card: " + amt));
order.completeOrder(new BigDecimal("49.99"));

// Comparator is Strategy too:
list.sort((a, b) -> b.price().compareTo(a.price())); // "highest price first" strategy
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `java.util.Comparator` passed to `Collections.sort` / `List.sort` — the textbook JDK Strategy.
- `java.util.function` interfaces (`Function`, `Predicate`, `Supplier`) used as pluggable behavior throughout streams and libraries.
- `ThreadPoolExecutor`'s `RejectedExecutionHandler` — `AbortPolicy`, `CallerRunsPolicy`, `DiscardPolicy` are interchangeable overflow strategies.

PITFALLS & CRITICISMS

If you only ever need one implementation, the interface is needless indirection. Writing a whole class for a one-line algorithm is ceremony that modern Java usually avoids — favor a functional interface plus a lambda or method reference unless the strategy needs its own state or multiple methods.

Staff-eng lens: Strategy is composition-based and needs no inheritance — that's precisely how it differs from Template Method (inheritance-based skeleton) and from State (the object switches its own strategy; here the client chooses and it stays fixed). Being fluent with this three-way distinction, and pointing out that modern Java collapses most Strategy usages into lambdas, is a strong signal at the staff level.

B Template Method

Intent (plain English): write the fixed skeleton of an algorithm once in a base class, and let subclasses fill in just the steps that vary.

Real-world analogy: a recipe card with fixed steps — preheat, cook, plate — but the filling or topping is customizable per dish. You can't skip a step or reorder them, but you can swap what goes inside.

PROBLEM IT SOLVES

Several classes implement almost the same algorithm, differing in only a couple of steps. Without a shared skeleton, that structure gets copy-pasted into every subclass — and the step *order* can silently drift out of sync between copies.

HOW IT WORKS

The base class defines one method (often `final`) that calls a fixed sequence of steps. Some steps already have a shared implementation; others are `abstract` (or have a default) and are left for subclasses to override. Subclasses can change *what* a step does but never the *order* of steps — this is the "Hollywood Principle": *don't call us, we'll call you*. The framework's base class is in control; your subclass just supplies the missing pieces.

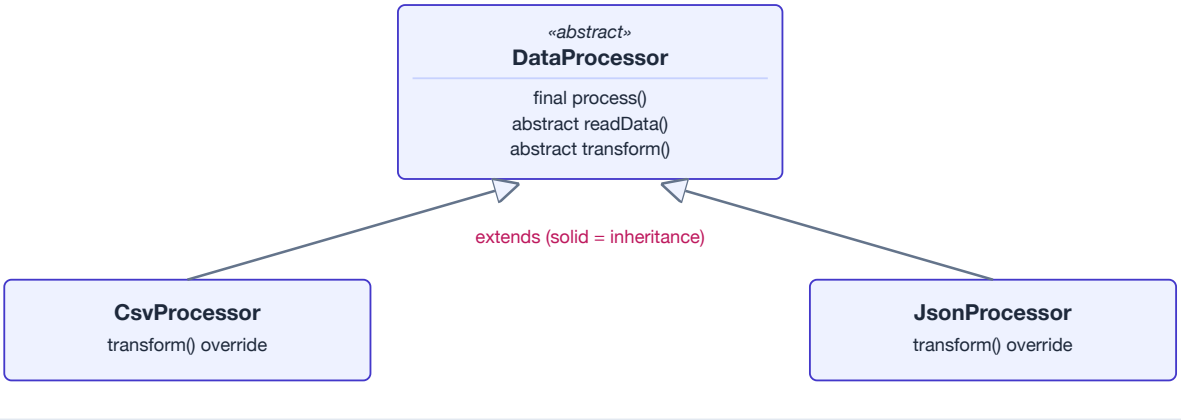

Figure — the base class fixes the algorithm's order; subclasses only override individual steps.

JAVA EXAMPLE

```
abstract class DataProcessor {
    final void process() {          // the skeleton -- order is fixed
        readData();
        transform();
        writeData();
    }
    abstract void readData();
    abstract void transform();
    void writeData() { System.out.println("Writing output..."); } // shared default
}

class CsvProcessor extends DataProcessor {
    void readData() { System.out.println("Reading CSV rows"); }
    void transform() { System.out.println("Trimming + parsing columns"); }
}

new CsvProcessor().process(); // caller never sees readData/transform/writeData directly
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `java.util.AbstractList` / `AbstractMap` — implement most collection behavior atop a handful of abstract primitives.
- Spring's `JdbcTemplate` and `RestTemplate` — open connection/execute/close is fixed; you supply the callback for the actual SQL or HTTP work.
- `InputStream.read(byte[])`, built on top of the single abstract `read()` method.
- `HttpServlet.service()` — fixed dispatch skeleton that calls your overridden `doGet`/`doPost`.

PITFALLS & CRITICISMS

Relies on inheritance, which is a tighter coupling than Strategy's composition. Subclasses can only vary what the base class explicitly allows via overridable steps — changing the skeleton itself means touching the base class and every subclass beneath it. Testing an individual step in isolation, without running the whole template, can also be awkward.

Staff-eng lens: reach for Template Method when the overall algorithm shape is genuinely fixed and shared and you already want an inheritance hierarchy. When you need runtime-swappable behavior, or you're wary of deep inheritance, prefer Strategy/composition instead — Template Method sits right on the "favor composition over inheritance" tension, and calling that out explicitly is what interviewers listen for.

B Visitor

Intent (plain English): add a new operation over a fixed set of classes without modifying those classes, by moving the operation into a separate visitor object.

Real-world analogy: a tax auditor visiting different account types — savings, checking, investment. The auditor knows how to handle each account type; the accounts themselves don't need to know anything about auditing.

PROBLEM IT SOLVES

You have a stable hierarchy of element classes (say, shapes in a document) but keep needing new, unrelated operations on them — area, render, export to XML. Putting every new operation directly on every element class bloats those classes and mixes unrelated concerns together.

HOW IT WORKS

Each element implements `accept(Visitor v)`, which simply calls `v.visit(this)`. The **Visitor** interface declares an overloaded `visit(...)` method per concrete element type. Because `accept()` is called with the element's actual compile-time type, and then the Visitor's overload resolution picks the matching `visit` method, two dispatches happen in sequence — element picks the visitor call, then the visitor's overload picks the right handler. That's **double dispatch**, something Java's single dispatch can't do directly.

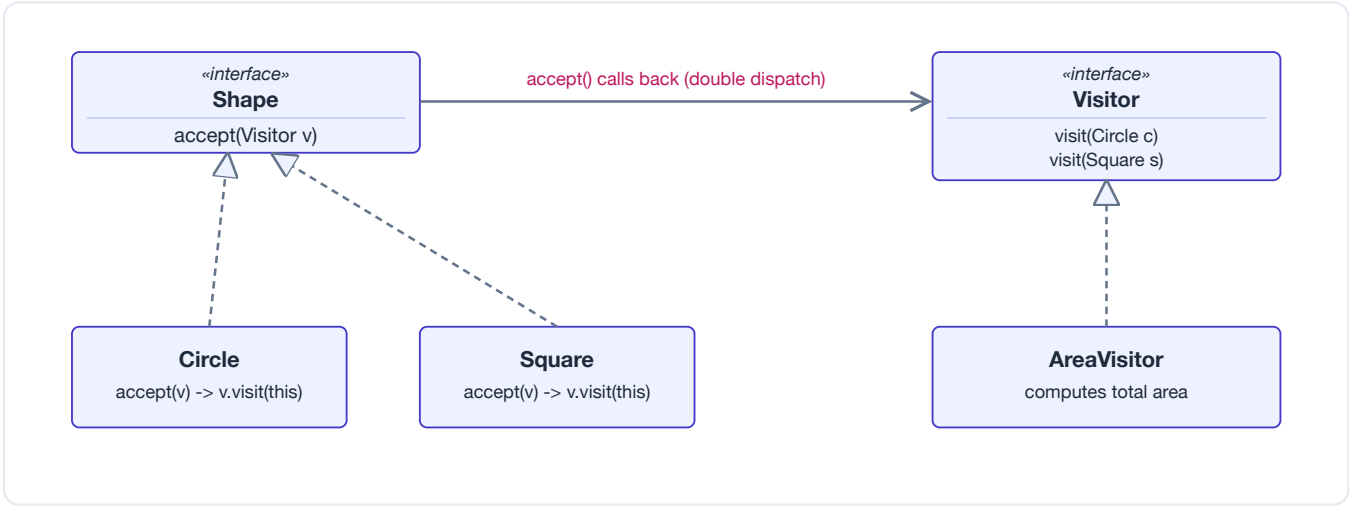

Figure — accept() and visit() together perform double dispatch to the right type-specific handler.

JAVA EXAMPLE

```
interface Shape { void accept(Visitor v); }
interface Visitor { void visit(Circle c); void visit(Square s); }

class Circle implements Shape {
    double radius;
    Circle(double r) { radius = r; }
    public void accept(Visitor v) { v.visit(this); } // dispatch #1
}
class Square implements Shape {
    double side;
    Square(double s) { side = s; }
    public void accept(Visitor v) { v.visit(this); }
}

class AreaVisitor implements Visitor {          // dispatch #2 picks the overload
    double total = 0;
    public void visit(Circle c) { total += Math.PI * c.radius * c.radius; }
    public void visit(Square s) { total += s.side * s.side; }
}

for (Shape shape : shapes) shape.accept(areaVisitor); // no new methods on Shape needed
```

IN THE REAL WORLD (JDK / SPRING / LIBRARIES)

- `javax.lang.model.element.ElementVisitor` and annotation-processing visitors.
- `java.nio.file.FileVisitor`, used by `Files.walkFileTree` to traverse directory trees.
- AST node visitors in compilers and linters — e.g. javac's `TreeVisitor`, ANTLR-generated visitors — for walking syntax trees.

PITFALLS & CRITICISMS

Adding one new element type (say, `Triangle`) forces you to add a matching `visit(Triangle)` overload to *every* existing Visitor implementation. Visitor inverts the usual tradeoff: it makes adding new *operations* easy but adding new *element types* hard — the opposite of ordinary polymorphism. Use it only when the element hierarchy is stable and it's the operations that keep growing.

Staff-eng lens: Visitor is the answer when someone asks "how do I add a new operation over a closed set of classes I shouldn't modify, without an instanceof/switch chain?" — but a strong staff-level answer names the tradeoff unprompted: it only pays off when the element hierarchy is closed/stable; if new element types appear often, plain polymorphism (or a switch expression with sealed interfaces, in modern Java) is usually simpler.

★ Cheat Sheet — All 23 Patterns

> One-line recall for each pattern: its intent and the trigger that should make you reach for it. In an interview, naming the pattern is table stakes — naming the *force* that justifies it (and the simpler alternative you rejected) is what reads as staff-level.

CREATIONAL — MAKING OBJECTS

Pattern	Intent (plain English)	Reach for it when…
Factory Method	Let subclasses decide which concrete class to create.	A class can't anticipate the exact type it must create; you want a hook to override.
Abstract Factory	Create whole families of related objects together.	Products must be used together and stay consistent (e.g. a UI theme's widgets).
Builder	Assemble a complex object step by step.	Many optional fields / telescoping constructors; want a readable, immutable result.
Prototype	Make new objects by copying a template instance.	Creating from scratch is costly, or the exact class is chosen at runtime.
Singleton	Exactly one instance, globally reachable.	Genuinely one shared resource — but prefer DI; Singleton is often an anti-pattern.

STRUCTURAL — COMPOSING OBJECTS

Pattern	Intent (plain English)	Reach for it when…
Adapter	Wrap one interface to look like another.	You must use an existing class whose interface doesn't match.
Bridge	Split abstraction from implementation so both vary.	A class explodes into a combinatorial matrix of subclasses.
Composite	Treat individual objects and trees uniformly.	You have part-whole hierarchies (files/folders, UI trees).
Decorator	Add responsibilities at runtime by wrapping.	You need many optional add-ons; subclassing would explode.
Facade	One simple front door to a complex subsystem.	You want to hide complexity and decouple clients from internals.
Flyweight	Share fine-grained objects to save memory.	You have huge numbers of similar objects with shareable state.
Proxy	A stand-in that controls access to a real object.	You need lazy loading, access control, remoting, or caching.

BEHAVIORAL — HOW OBJECTS INTERACT

Pattern	Intent (plain English)	Reach for it when…
Chain of Responsibility	Pass a request down a chain until someone handles it.	Multiple handlers, decided at runtime (filters, middleware).
Command	Turn a request into an object.	You need undo/redo, queuing, logging, or scheduling of actions.
Interpreter	Represent a small language and evaluate it.	You have a simple, stable grammar to evaluate repeatedly.
Iterator	Walk a collection without exposing its structure.	You want uniform traversal independent of the container.
Mediator	Centralize how a set of objects interact.	Many objects are tangled in many-to-many references.
Memento	Snapshot and restore state without breaking encapsulation.	You need checkpoints / undo without exposing internals.
Observer	Notify dependents automatically on state change.	One change must fan out to many loosely-coupled listeners.
State	Change behavior as internal state changes.	Big conditionals switch on a state field; model a state machine.
Strategy	Swap interchangeable algorithms at runtime.	You have several ways to do one thing, chosen dynamically.
Template Method	Fix an algorithm's skeleton; defer steps to subclasses.	Several variants share a flow but differ in a few steps.
Visitor	Add operations to a structure without changing its classes.	Stable object structure, but operations keep getting added.

THE ONE THING TO REMEMBER

Patterns are a **vocabulary**, not a checklist. Their real value is naming a recurring design so a team can discuss it in one word. The two forces behind almost all of them are the same: **program to an interface, not an implementation**, and **favor composition over inheritance**. Apply a pattern to remove a *specific* pain (change is hard, coupling is tight, a class does too much) — never because the pattern is elegant. The most senior move in a design review is often deleting a pattern that isn't earning its complexity.

STAFF-ENGINEER INTERVIEW ANGLES

- **Strategy vs State vs Template Method:** all vary behavior — know the difference (interchangeable algorithm vs state-machine transition vs fixed skeleton with overridable steps).
- **Factory Method vs Abstract Factory vs Builder:** one product via subclass hook vs families of products vs step-by-step assembly.
- **Decorator vs Proxy vs Adapter:** all wrap — add behavior vs control access vs convert interface.
- **Singleton:** be ready to argue why it's often an anti-pattern (hidden global state, hard to test) and reach for dependency injection instead.
- **Over-engineering:** show judgment by rejecting a pattern when a plain function, enum, or lambda is simpler. Modern Java (lambdas, records, sealed types) makes several classic patterns nearly free or unnecessary.